Al-Quds University

Faculty of Engineering

Department of Computer Engineering

Project name:

Constructs

An AI-Assisted Construction Intelligence and Collaboration Platform for Design-to-Execution Integration

By:

Anwar Roumanen

Supervisor:

Prof. Dr. Eng. Salaheddin Odeh

This software project is submitted in partial fulfilment of the requirements for the degree of BSc in Computer Engineering from Al-Quds University.

Palestine, 2026

Dedication

[Dedication to be written by the author.]

Acknowledgement

[Acknowledgement to be written by the author, for example thanking the supervisor, Prof. Dr. Eng. Salaheddin Odeh, and the Department of Computer Engineering.]

Abstract

Construction projects are delivered by owners, consulting engineers, contractors and specialist disciplines that exchange large volumes of interdependent information: building models, drawings, specifications, schedules, site evidence and decisions. Because this information usually lives in separate tools and conversations, a change in the design is often discovered on site rather than in the plan. This report presents **Constructs**, a web and mobile platform that keeps design information and project execution in one permission-controlled record and provides concrete mechanisms that connect them.

Constructs is implemented as a FastAPI backend with PostgreSQL and the pgvector extension, a durable job worker, a React web client and a Flutter mobile client. Its design-to-execution mechanisms are: IFC model processing that extracts spatial structure and elements and proposes tasks and milestones; revision comparison by IFC GlobalId with impact suggestions on linked or same-discipline tasks; interference candidates that can be turned into issues; retrieval-augmented question answering over authorised documents whose citations are built from retrieved records; and a bilingual voice workflow that converts spoken site reports into confirmed task updates. A single permission-resolution function governs every one of these paths, with ceilings that keep external parties away from certification and access-governance authority.

The automated test suites were executed for this report: the backend suite ran 2798 tests with 2796 passing, 1 environment-dependent failure and 1 intentional skip; the web suite passed 294 of 310 tests, the 16 remaining being timeouts that passed on a serial re-run; and the mobile suite passed all 141 tests. These results verify behaviour at the unit and API-integration level; end-to-end user workflows, live AI-provider behaviour, performance and usability have not yet been systematically evaluated, and the report states these limits explicitly.

Keywords: construction management, design-to-execution integration, BIM, IFC, role-based access control, critical path method, retrieval-augmented generation.

Table of Contents

Chapter 1: Introduction 1

1.1 Project Overview 1

1.2 Objectives and Scope 2

1.3 Problem Statement 3

1.4 Significance of the Project 4

1.5 Project Constraints and Assumptions 4

Chapter 2: Literature Review and Theoretical Framework 6

2.1 Review of Relevant Literature 6

2.1.1 Fragmentation and collaboration in construction 6

2.1.2 BIM, IFC and design-to-execution integration 6

2.1.3 Large language models and document-grounded answers 7

2.1.4 Access control and web security 7

2.2 Theoretical Framework 7

2.2.1 Critical path method 7

2.2.2 Permission resolution with ceilings 8

2.2.3 Grounded answering 8

2.2.4 Software architecture principles 8

2.3 Existing Solutions and Comparative Analysis 8

2.4 Research Gap and Positioning of Constructs 9

Chapter 3: System Requirements and Analysis 11

3.1 Stakeholder Analysis 11

3.2 Functional Requirements 12

Authentication, authorization and administration 12

Project and stakeholder management 13

Tasks, dependencies and schedules 13

Documents and engineering information 13

BIM/IFC processing and coordination 13
AI-assisted analysis and retrieval 14
Site execution and reporting 14
Communication, notification and traceability 15
3.3 Non-Functional Requirements 15
3.4 System Models and Diagrams 17
3.4.1 Use case diagrams 17
3.4.2 Key use case descriptions 19
3.4.3 Activity diagram - task execution and review 20
3.4.4 State machine diagrams 22
Chapter 4: System Design and Architecture 25
4.1 High-Level Architecture 25
4.2 Detailed Design, Data Design, and Interface Design 27
4.2.1 Backend structure 27
4.2.2 Data design 27
4.2.3 Interface design 30
4.3 Data Flow and Process Models 31
4.4 End-to-End Scenarios 38
4.4.1 Scenario 1 - from a contract PDF to a cited answer 38
4.4.2 Scenario 2 - from a model revision to affected work 38
4.4.3 Scenario 3 - from a voice report on site to an updated plan 38
4.5 Design Patterns and Justifications 43
Chapter 5: Implementation and Testing 44
5.1 Development Environment and Tools 44
5.2 Implementation Details, Key Algorithms, and Code Structure 44
5.2.1 Permission resolution 45
5.2.2 Critical path calculation 45
5.2.3 Durable job queue 46

5.2.4 File ingestion 46
5.2.5 Retrieval-augmented answering 47
5.2.6 IFC processing, revision comparison and interference 47
5.2.7 Model-to-execution alignment score 48
5.2.8 Voice assistant and agents 49
5.3 Version Control and Change Management 49
5.4 Testing Strategy 49
5.4.1 Unit Testing 49
5.4.2 Integration Testing 49
5.4.3 System Testing 50
5.5 Test Results and Analysis 50
Chapter 6: Business Analysis and Project Evaluation 52
6.1 Business Model Canvas 52
6.1.1 Key Partners 52
6.1.2 Key Activities 52
6.1.3 Value Propositions 52
6.1.4 Customer Relationships 52
6.1.5 Customer Segments 53
6.1.6 Key Resources 53
6.1.7 Channels 53
6.1.8 Cost Structure 53
6.1.9 Revenue Streams 53
6.2 Cost Structure and Budget Analysis 53
6.3 Impact Analysis 54
6.3.1 Social Impact 54
6.3.2 Economic Impact 54
6.3.3 Environmental Impact 54
6.4 Project Evaluation and Lessons Learned 54

6.4.1 Achievement of objectives 54
6.4.2 Strengths 55
6.4.3 Limitations 55
6.4.4 Future work 56
6.4.5 Lessons learned 56
Appendices 57
Appendix A: Glossary of Terms 57
Appendix B: References 57
Appendix C: Supporting Materials 60
Appendix D: Requirements Traceability Summary 61
Appendix E: Test Execution Details 62

List of Figures

Figure 1.1: Design-to-execution mechanisms implemented in Constructs 2
Figure 3.1: System context of Constructs 11
Figure 3.2: Use cases - project execution and collaboration 18
Figure 3.3: Use cases - engineering information, AI assistance and administration 19
Figure 3.4: Activity diagram - task execution and review 21
Figure 3.5: State machine - task status 22
Figure 3.6: State machine - IFC model version processing status 23
Figure 3.7: State machine - ingested file status (PARTIAL is a usable outcome with reduced capability) 24
Figure 4.1: High-level architecture 25
Figure 4.2: Backend component diagram 26
Figure 4.3: Deployment diagram (Docker Compose) 27
Figure 4.4: ER diagram - identity, organisation and RBAC 28
Figure 4.5: ER diagram - projects, parties, tasks and field records 29
Figure 4.6: ER diagram - documents, ingestion, retrieval chunks and jobs 29
Figure 4.7: ER diagram - IFC model groups, versions, elements and analysis results 30
Figure 4.8: Activity diagram - effective permission resolution 32
Figure 4.9: Sequence diagram - login, authorised request and token refresh 33
Figure 4.10: Sequence diagram - durable background job processing 34
Figure 4.11: Level-1 data flow diagram - file ingestion and document question answering 35
Figure 4.12: Activity diagram - IFC upload, processing and coordination 36
Figure 4.13: Activity diagram - revision comparison and impact suggestions 37
Figure 4.14: Sequence diagram - retrieval-augmented document question answering 40
Figure 4.15: Sequence diagram - voice command to confirmed action 41
Figure 4.16: Sequence diagram - notification, realtime and push fan-out 42
Figure 5.1: Class diagram - retrieval and answering services 47

List of Tables

Table 2.1: Capabilities documented by vendors, compared with Constructs (sources accessed 24 Oct 2026) 9
Table 5.1: Verified technology choices (versions from the dependency manifests) 44
Table 5.2: Backend test results by functional area (pytest)..... 50
Table 5.3: Test outcomes by status across all suites 51
Table A.1: Functional requirements with representative tests 61

List of Listings

Listing 5.1: Effective permission resolution (abridged from services/authorization.py)..... 45
Listing 5.2: Dependency-type conversion and forward pass (abridged from services/cpm_service.py) 46
Listing 5.3: Claiming the next job under a fleet-wide ceiling (simplified from services/jobs/queue.py) 46
Listing 5.4: Revision comparison and impact logic (pseudocode of ifc_processing_service.compare_versions) 48

Chapter 1: Introduction

This chapter introduces Constructs, defines the problem it addresses and why that problem matters, states the project's objectives and scope, and lists the constraints under which it was developed.

1.1 Project Overview

Constructs is a construction intelligence and collaboration platform for a consulting engineering office that supervises building projects together with its client and its contractors. It began as the "Smart Construction Platform" proposal and is referred to in parts of its user interface as StructIQ; this report uses the official project name, Constructs, throughout.

The platform's central idea is that design information and execution information should not live in separate worlds. A building model revision, a clause in a specification or a design change should be connected to the tasks, issues and milestones it affects, and a report from the site should update the plan it refers to. Figure 1.1 summarises the mechanisms that Constructs implements for this purpose; each is described in detail in Chapters 4 and 5.

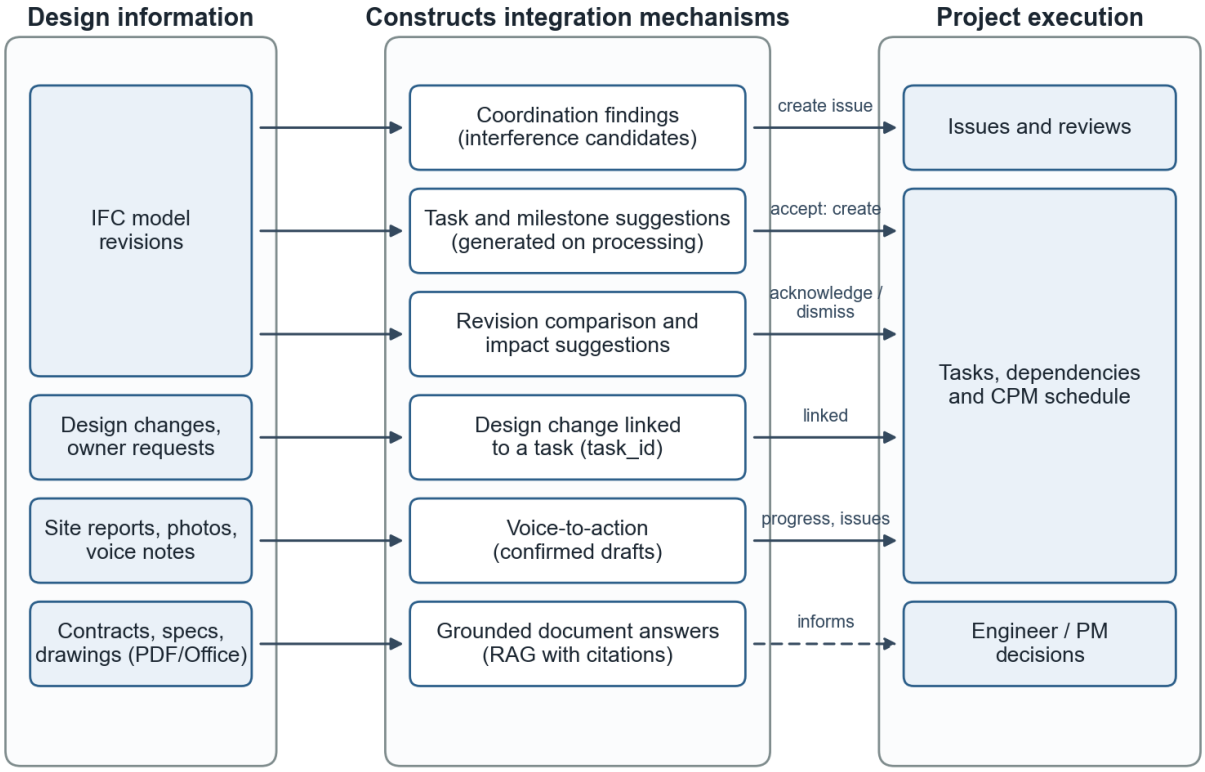


Figure 1.1: Design-to-execution mechanisms implemented in Constructs

The system is organised as a monorepo with four runtime parts: a REST API built with FastAPI [1]; a PostgreSQL 15 database with the pgvector extension [2]; a background worker that executes long-running jobs such as IFC parsing and document indexing; and two clients - a React 19 web application [3] for office work and a Flutter application [4] for field work. Both clients support English and Arabic with right-to-left layout.

The implementation is substantial for a graduation project. At the time of writing, the backend contains 251 Python modules (about 58.6k lines), exposes 337 HTTP route handlers and declares 80 database tables managed by 71 Alembic migrations; the web client has 289 TypeScript files and the mobile client 92 Dart files. These figures were counted from the working tree and are reported to convey scale, not quality.

1.2 Objectives and Scope

The objectives below restate the proposal's goals in a form that can be checked against the implementation. Chapter 6 evaluates each of them.

- **O1 - One connected project record:** projects, parties, members, tasks, milestones, documents, models, site reports, issues, design changes and messages are stored and related in one system.
- **O2 - Configurable, safe access:** an office defines its own roles and permissions, and external parties see only what is shared with them, without code changes.
- **O3 - Schedule control:** tasks with all four dependency types and lags, critical path calculation and delay analysis.
- **O4 - Design-to-execution links:** IFC models are processed, compared across revisions and connected to tasks, issues and milestones through reviewable suggestions.
- **O5 - Document-grounded assistance:** questions about project documents are answered only from authorised sources, with citations a reader can check.
- **O6 - Field feedback:** site staff report progress, evidence and problems from a phone, including by voice in Arabic or English, with confirmation before any change.
- **O7 - Traceability:** important actions, AI-initiated changes and decisions are recorded and attributable to a person.

In scope are the web and mobile clients, the API, the database, the job worker and the AI-assisted workflows above, deployed with Docker Compose. **Out of scope** are DWG/DXF interpretation (such files are stored but not parsed), payment, ERP and accounting integrations, a production cloud deployment, iOS release builds, and autonomous AI behaviour without a human trigger.

1.3 Problem Statement

Construction is one of the largest sectors of the economy, yet its productivity has grown slowly compared with other industries; fragmentation, opacity and misaligned incentives between parties are among the reasons repeatedly identified [5]. Research on systems integration in architecture, engineering and construction describes the same structural difficulty from a technical angle: many organisations and tools must create and use shared information across the project lifecycle [6], and even teams that adopt BIM face persistent collaboration barriers [7].

The proposal for this project observed these problems in local practice. Project information is scattered across files, messaging groups and personal phones; records that belong together - a task, the photograph that proves it, the drawing it implements and the decision that changed it - are not linked; permission checks are inconsistent, so external parties are either over- or under-informed; and when a design changes, its effect on scheduled work is recognised late.

The problem this project addresses is therefore stated as: *how can design information (models, documents, design changes) and execution information (tasks, schedules, site evidence, issues) be held in one permission-controlled system with explicit, reviewable links between them, so that design changes and site conditions reach the plan quickly, without giving AI or external parties authority they should not have?* Section 2.4 refines this into a specific research gap after the literature and existing products have been examined.

1.4 Significance of the Project

For a consulting office, the practical significance is a single place where a project manager can see whether the current model, the current documents and the current plan agree, and where an external contractor can participate without being able to certify their own work. For the field, it shortens the path from an observation on site to an update in the plan.

Technically, the project is significant because it combines three bodies of knowledge - BIM and IFC [8], [9], critical path scheduling [10] and role-based access control [11] - with a disciplined use of large language models. Hallucination is a documented property of generative models [12]; Constructs therefore uses retrieval-augmented generation [13] with safeguards enforced in code, and uses deterministic rules wherever a decision can be computed rather than interpreted.

1.5 Project Constraints and Assumptions

- **Team and time:** one student developer over one academic term (the Git history shows a single author identity under two names); depth in core workflows was prioritised over breadth.
- **Infrastructure:** no funded hosting; development on a personal Windows workstation with Docker Desktop, so the deployment is a single-host Docker Compose stack.
- **External services:** AI features depend on the paid OpenAI API and push notifications on Firebase; both are optional and behind configuration switches, and the settings validator refuses incoherent configurations.
- **Data:** no access to real client project data; development and testing use generated fixtures and an optional demo seed.
- **Standards:** engineering models are exchanged as IFC, parsed with IfcOpenShell [14]; proprietary DWG is out of scope.
- **Assumption - field conditions:** site users have Android or iOS phones with intermittent connectivity; the server remains the source of truth.

- **Assumption - AI output can be wrong:** every AI proposal is confirmed by a person and re-authorised by the backend before it changes data.

Chapter 2: Literature Review and Theoretical Framework

This chapter reviews the research that frames the problem, explains the theory used in the implementation, compares established products using their own documentation, and derives the specific gap that Constructs addresses.

2.1 Review of Relevant Literature

2.1.1 Fragmentation and collaboration in construction

Project management standards describe delivery as the coordinated planning, measurement and control of work, stakeholders and risk [15]. Construction makes this coordination unusually hard because each project assembles a temporary network of independent organisations. Shen et al. review two decades of systems-integration and collaboration technology in architecture, engineering, construction and facilities management, and frame the goal as integrating "people, processes, business systems, and information" across the project lifecycle [6]. More recently, Oraee et al. analysed 73 journal articles on BIM-based construction networks and concluded that, although BIM's value depends on collaboration among dispersed, multi-organisation teams, members still struggle to collaborate [7]. The lesson for this project is that adding a model viewer or a document store does not by itself connect people and decisions; the links between records must be designed.

2.1.2 BIM, IFC and design-to-execution integration

Building information modelling represents a facility as structured, object-based data rather than drawings [8]. Azhar summarises the benefits - visualisation, clash detection, quantity take-off, better coordination - together with risks around ownership, interoperability and process change [16]. The vendor-neutral exchange schema is the Industry Foundation Classes (IFC), standardised as ISO 16739-1 [9]; it defines spatial structure (site, building, storey, space), physical elements, properties and quantities, and gives each object a globally unique identifier (GlobalId) - the property Constructs relies on to compare revisions.

Connecting models to execution is an active research area. 4D BIM adds time to the model to anticipate delays and optimise workflows; a 2025 systematic review notes that dedicated reviews of 4D BIM remain scarce and identifies lean integration, predictive strategies and ontology frameworks as key strategies [17]. Other work generates and optimises construction schedules directly from BIM data [18]. Coordination research also warns that geometric clash detection

after the fact is only part of the answer, and argues for coordinated processes that avoid clashes [19]. Constructs follows this view: interference results are presented as candidates for review, and model changes become suggestions that a responsible person accepts or dismisses.

2.1.3 Large language models and document-grounded answers

Reviews of generative AI in construction identify opportunities across the project lifecycle together with challenges that limit adoption, and validate prototypes for specific tasks [20]; recent surveys extend this to multimodal models in construction [21]. Zheng and Fischer propose a prompt-based virtual assistant for searching BIM information [22], illustrating the interest in natural-language access to engineering data. The main technical obstacle is reliability: language models can produce fluent but unsupported statements [12]. Retrieval-augmented generation (RAG) conditions the answer on passages retrieved from an external corpus [13], and later surveys describe how chunking, retrieval and post-processing improve such pipelines [23]. Dense retrieval relies on vector embeddings [24] and approximate nearest-neighbour indexes such as HNSW [25].

2.1.4 Access control and web security

Role-based access control assigns permissions to roles and roles to users, simplifying administration and supporting least privilege [11]; the NIST model formalised core RBAC, hierarchies and separation-of-duty constraints [26]. Broken access control is ranked first in the OWASP Top 10 [27]. In a multi-party construction project two extensions are needed: authority that depends on the project (the same engineer may lead one project and be absent from another), and constraints no configuration may override - most importantly, a contractor must not be able to approve its own work.

2.2 Theoretical Framework

2.2.1 Critical path method

The critical path method models a project as a directed acyclic graph of activities with durations and precedence constraints [10]. A forward pass computes each activity's earliest start (ES) and earliest finish (EF = ES + d); a backward pass computes latest start (LS) and latest finish (LF); total float is TF = LS - ES, and zero-float activities form the critical path. With predecessor duration d(p), successor duration d(s) and lag L, each precedence type becomes a constraint on the successor's start:

- Finish-to-start (FS): ES(s) >= ES(p) + d(p) + L, the default construction relation;
- Start-to-start (SS): ES(s) >= ES(p) + L;

- Finish-to-finish (FF): ES(s) >= ES(p) + d(p) + L - d(s);
- Start-to-finish (SF): ES(s) >= ES(p) + L - d(s).

This conversion lets one forward and one backward pass handle all four types; Section 5.2 shows the implementation.

2.2.2 Permission resolution with ceilings

The authorization model can be written as a function P(u, p) returning the permissions of user u in project p: the union of the user's office-role and project-role permissions, modified by explicit allow and deny overrides (user-wide first, then project-specific), and finally restricted by ceilings that overrides cannot lift. Applying the ceilings last is what turns a configuration mistake into a refusal rather than a security breach.

2.2.3 Grounded answering

An answer is produced as a = G(q, R(q, D(u))), where R retrieves the k passages nearest to the question embedding by cosine distance, 1 - (x . y)/(||x|| ||y||), and D(u) is the part of the corpus user u may read. Restricting the corpus before retrieval, rather than filtering afterwards, guarantees that unauthorised text never reaches the model.

2.2.4 Software architecture principles

The API follows the REST style [28]; login uses the OAuth 2.0 password form [29], tokens are JSON Web Tokens [30] and passwords are hashed with Argon2 [31]. The backend is layered and uses enterprise patterns such as Service Layer and Unit of Work [32] and object-oriented patterns such as Strategy [33]. Asynchronous work relies on PostgreSQL row locking with SKIP LOCKED [34], advisory locks [35] and LISTEN/NOTIFY [36]; live updates use server-sent events [37].

2.3 Existing Solutions and Comparative Analysis

Four established products were selected because each covers part of Constructs' scope: Procore and Autodesk's construction platform (now marketed under the Forma name) as integrated construction-management suites, Oracle Aconex for document control and correspondence, and Primavera P6 as a dedicated CPM scheduler. Capabilities were checked against the vendors' own help or product pages on 24 October 2026. "Documented" means the consulted page describes the capability; "Not found" means it was not found on the consulted pages, which is not evidence that the product lacks it.

Table 2.1: Capabilities documented by vendors, compared with Constructs (sources accessed 24 Oct 2026)

Capability	Procore	Autodesk (ACC / Forma)	Oracle Aconex	Primavera P6	Constructs
Multi-party project workflows	Documented [38]	Documented [39]	Documented [40]	Documented [41]	Implemented
CPM scheduling	Scheduling shown [38]	Schedule insights [39]	Via Primavera integration [40]	Core capability [41]	Implemented
Model coordination / clashes	Coordination issues in model viewer [42]	Automatic clash detection incl. IFC [43]	BIM model coordination [40]	Not found	Interference candidates (bounding boxes)
Q&A over project documents with source links	Assist: answers with links, respects permissions [44]	Autodesk Assistant on specifications (vendor blog) [45]	Not found	Not found	RAG with record-built citations
Model revision changes linked to scheduled tasks	Not found	Not found	Not found	Not found	Impact suggestions by element link or discipline
Arabic interface and spoken-Arabic input	Not found	Not found	Not found	Not found	Implemented
Deployment	SaaS	SaaS	SaaS	On-premise / cloud	Self-hosted (Docker)

The established products are clearly stronger in maturity, scale, integration ecosystems and depth of each individual capability. Autodesk documents automatic clash detection across RVT, DWG, NWC and IFC models with a workflow to turn clashes into issues [43]; Procore documents an AI assistant that answers questions from RFIs, submittals, drawings, specifications and documents, links to its sources and respects user permissions [44]. Constructs does not match these products on breadth, and its interference detection is a simpler bounding-box method.

Two observations nevertheless matter for this project. First, document-grounded AI with source links is becoming a standard expectation, which supports the design choice of retrieval with verifiable citations rather than free generation. Second, on the pages consulted, none of the products documents an explicit mechanism that turns a difference between two model revisions into a reviewable suggestion on the scheduled tasks it affects. Such functionality may exist through add-ons or integrations; the claim here is limited to what the vendors' own documentation states.

2.4 Research Gap and Positioning of Constructs

The literature and the product survey separate five levels that should not be confused:

1. **General problem.** Construction information is fragmented across organisations and tools, and collaboration remains difficult even with BIM [6], [7].

2. **Gap.** Research links BIM to schedules mainly through 4D simulation and schedule generation [17], [18], and commercial suites document coordination and AI search as separate tools. A lightweight, auditable link from a model revision to the specific execution records it may affect - together with permission-aware, citation-grounded document answers - is not documented as one workflow in the sources examined. This is a practical integration gap, not a claim that no such system exists.
3. **Specific problem for Constructs.** Within one consulting office's projects, keep model revisions, documents, tasks, issues and site evidence connected, and make every cross-link reviewable by an authorised person.
4. **Contribution.** An implemented platform in which GlobalId-based revision comparison produces impact suggestions on linked or same-discipline tasks, model processing proposes tasks and milestones, interference candidates become issues, document answers carry record-built citations, voice reports become confirmed updates, and all paths share one permission function with external-party ceilings.
5. **Limitations.** The mechanisms are verified by automated unit and integration tests, not by field trials; the discipline fallback is a heuristic; interference detection uses bounding boxes; no user study, performance measurement or live-AI evaluation has been carried out (Section 6.4).

Chapter 3: System Requirements and Analysis

Requirements were derived from the original proposal and checked against the implemented code. Each requirement carries a status: **Implemented** means code and automated tests exist; **Partial** means part of it is missing or in transition; **disabled by default** means the capability exists but is switched off. The traceability workbook (Constructs_Traceability_Matrix.xlsx) maps every requirement to source files and tests.

3.1 Stakeholder Analysis

Figure 3.1 places Constructs in its environment. Internal users belong to the consulting office; external users belong to other organisations on a project and are always constrained by the party they represent. OpenAI, Firebase Cloud Messaging and SMTP are optional external services; the platform runs without them, with the corresponding features reported as unavailable.

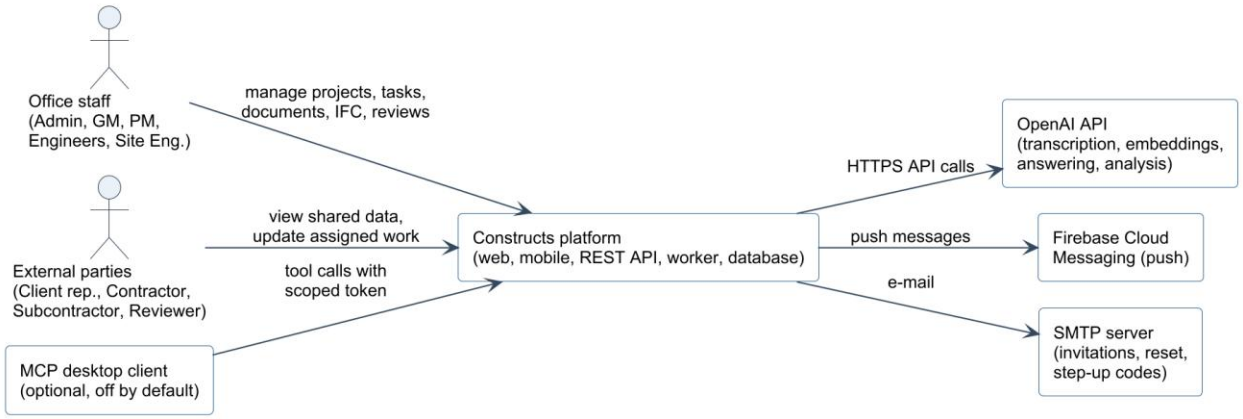


Figure 3.1: System context of Constructs

The stakeholders correspond to the role templates seeded by the system. An office may rename or re-permission any template except the undeletable Office Administrator, so the list below describes defaults rather than fixed roles.

- **Office administrator (internal)** - accounts, roles and access governance; does not take part in project work by default.
- **Office leadership (internal: General Manager, Technical Director)** - oversight of every project and technical approval authority such as approving design changes.
- **Project manager (internal)** - plans the project, assigns members and tasks, edits the schedule and reviews work.

- **Engineers (internal: senior, engineer, BIM engineer, CAD technician, surveyor)** - carry out assigned tasks, report progress and issues, upload models and documents; drafting roles hold no review authority by default.
- **Site engineer (internal)** - reports from the field with site reports, photographs and voice.
- **Document control and administrative staff (internal)** - maintain the document library.
- **Client representative (external)** - follows progress, spending and evidence and raises owner requests.
- **Main contractor and subcontractors (external)** - update assigned work and submit evidence; can never certify work or grant access.
- **External reviewer (external)** - reviews shared material without certification authority.

3.2 Functional Requirements

Thirty functional requirements are grouped by functional area. Twenty-seven are implemented and tested; FR-08 and FR-19 are partial, and FR-23 is implemented but disabled by default. Requirements from the proposal that were not built are listed at the end of this section rather than presented as capabilities.

Authentication, authorization and administration

- **FR-01 - Sign-in and sessions.** Users sign in with e-mail and password and receive a 30-minute access token and a single-use, rotating 7-day refresh token; logout and password reset are supported; self-registration is disabled. *Status: Implemented.*
- **FR-02 - Step-up verification.** A first-login password change is forced, and sensitive operations require an e-mailed one-time code verified within a short window. *Status: Implemented.*
- **FR-03 - Configurable roles.** An office administrator manages accounts, roles, role permissions, disciplines and memberships, starting from seeded role templates, without code changes. *Status: Implemented.*
- **FR-04 - Per-person exceptions.** An administrator grants or denies a single permission to one person, everywhere or on one project, and can see which rule decided each permission. *Status: Implemented.*
- **FR-05 - External-party isolation.** External participants see only project-scoped data explicitly shared with their party and can never hold certification or access-governance permissions. *Status: Implemented.*

Project and stakeholder management

- **FR-06 - Projects.** Authorised users create, edit and delete projects, with a deletion-impact preview and a configurable consultant approval workflow. *Status: Implemented.*
- **FR-07 - Members and parties.** Project managers assign members with project roles and organise external parties (client, main contractor, subcontractors). *Status: Implemented.*
- **FR-08 - Dashboards and budget view.** Users see project summaries, an owner dashboard and action-centre counts; projects hold budget total and spent values, and cost validation requests are reviewed with variance and risk level. *Status: Partial.*

Tasks, dependencies and schedules

- **FR-09 - Task management.** Project managers create, assign, prioritise, reorder and edit tasks with planned and actual dates, durations, disciplines and milestones. *Status: Implemented.*
- **FR-10 - Task execution and review.** A task starts only from TODO when predecessors are done and no blocker is open; assignees report progress and blockers, submit for review, and reviewers approve, reject with corrections or request clarification. *Status: Implemented.*
- **FR-11 - Dependencies and CPM.** Tasks are linked by FS, SS, FF or SF dependencies with lag; cycles are rejected; the system provides Gantt data, a connected critical path, delay analysis and schedule shifting. *Status: Implemented.*

Documents and engineering information

- **FR-12 - Document library.** Members upload, search, download and delete documents through authenticated endpoints and share individual documents with an external party. *Status: Implemented.*
- **FR-13 - File ingestion.** Uploaded files (PDF, DOC/DOCX, XLS/XLSX, IFC, images, ZIP, TXT) are validated, checksummed, de-duplicated, classified and processed asynchronously; DWG/DXF files are stored but not parsed. *Status: Implemented.*
- **FR-14 - Design changes and requests.** Engineers propose design changes with cost and schedule impact, linked to a task; reviewers approve, reject or mark them implemented; clients raise owner requests. *Status: Implemented.*

BIM/IFC processing and coordination

- **FR-15 - IFC processing.** IFC files are uploaded into model groups and versions and parsed in an isolated process into spatial hierarchy, elements, properties and quantities, with quality checks. *Status: Implemented.*

- **FR-16 - Model-derived suggestions.** Processing proposes construction tasks per element class and per-storey milestones, flagged for duplicate risk; accepting a suggestion creates the task or milestone. *Status: Implemented.*
- **FR-17 - Revision comparison and impact.** Two versions of one model group are compared by GlobalId into added, removed, modified and moved elements; impact suggestions are created on linked items or same-discipline tasks and are acknowledged or dismissed. *Status: Implemented.*
- **FR-18 - Coordination findings.** Structural/service interference candidates are computed from element bounding boxes and can be turned into project issues, ignored or marked false positive. *Status: Implemented.*
- **FR-19 - 3D viewing.** Viewer geometry is generated server-side per version for display in a 3D web viewer. *Status: Partial.*

AI-assisted analysis and retrieval

- **FR-20 - Grounded document answers.** Questions are answered only from documents and files the user may read, with citations to document and page, and a not-found answer when the context is insufficient. *Status: Implemented.*
- **FR-21 - Question routing.** A question is routed deterministically to structured project data, IFC data or document retrieval, defaulting to documents when unsure. *Status: Implemented.*
- **FR-22 - Rule-based insights and agents.** Deterministic analyzers produce evidence-carrying findings: five agents (site report, document consistency, revision impact, RFI, schedule risk) on demand, and a model-to-execution alignment score. *Status: Implemented.*
- **FR-23 - Event-driven agent analysis.** Agents run automatically when domain events occur, under the project manager's authority. *Status: Implemented, disabled by default.*

Site execution and reporting

- **FR-24 - Site reports and evidence.** Site staff submit daily reports and field evidence with directional photographs; authorised engineers verify or return them; photos are archived by category. *Status: Implemented.*
- **FR-25 - Voice-to-action.** Spoken Arabic or English is transcribed and analysed; questions are answered; proposed changes become drafts executed only after confirmation and a fresh permission check. *Status: Implemented.*
- **FR-26 - Issues, visits and reminders.** Issues with severity and status are linked to tasks; site visits are scheduled with participants; reminder rules send automatic reminders. *Status: Implemented.*

Communication, notification and traceability

- **FR-27 - Messaging.** Members exchange direct, group, project and contextual messages with acknowledgement, response tracking and forwarding. *Status: Implemented.*
- **FR-28 - Notifications.** Users are notified in-app, by push (Firebase) and by e-mail; the web client receives live update hints over server-sent events. *Status: Implemented.*
- **FR-29 - Audit and AI history.** Important actions are recorded in audit logs and domain events; AI-initiated changes are stored with before and after state and can be reverted. *Status: Implemented.*
- **FR-30 - Bilingual interface.** Web and mobile interfaces are available in English and Arabic with right-to-left layout. *Status: Implemented.*

Partial and disabled requirements. FR-08: budget fields and cost-validation review exist, but the proposal's expense records linked to evidence, with an owner-facing spending breakdown, are not implemented. FR-19: geometry generation is implemented and tested, and the last commit contains a Three.js viewer, but the current working tree deletes the viewer component and its dependency (an uncommitted change), so 3D viewing is in transition. FR-23: event-driven analysis is tested but switched off (`AGENT_AUTO_ANALYSIS_ENABLED=false`).

Proposal items not implemented. Expense tracking linked to evidence (above), DWG interpretation (explicitly excluded in the proposal), detection of contradictions between two documents (the document-consistency agent states that this is not implemented), and an additional Telegram notification channel, which is declared in an enumeration but has no provider.

3.3 Non-Functional Requirements

No numerical quality targets were specified in the proposal, and none are invented here. The non-functional requirements below are grouped by the strength of the available evidence.

Supported by implementation and tests:

- **NFR-01 (Security).** Passwords hashed with Argon2; typed tokens (access, refresh, SSE ticket) that cannot substitute for one another; single-use refresh rotation.
- **NFR-02 (Security).** Every subsystem resolves permissions through one function; ceilings are applied after overrides; the server re-checks every operation regardless of client state.
- **NFR-03 (Security).** Database-backed rate limits on login, one-time-code sending and verification, and MCP tool calls, correct across several API workers.
- **NFR-04 (Confidentiality).** Documents, IFC sources, geometry and audio are stored privately and streamed only through authenticated endpoints.

- **NFR-05 (Reliability).** Background jobs are enqueued in the causing transaction, de-duplicated, retried with exponential backoff and reclaimed after a stale heartbeat.
- **NFR-06 (Reliability).** Secondary products (geometry, agents, push, indexing) cannot fail the primary operation that triggered them.
- **NFR-07 (Consistency).** Notifications and realtime events are written in the caller's transaction and delivered only after commit.
- **NFR-15 (AI governance).** AI proposes, a person confirms, the backend authorises and executes; provider calls and AI actions are recorded.

Supported by implementation:

- **NFR-14 (Configuration safety).** The settings validator refuses to start when a feature is enabled without its required configuration.

Implemented; limits configured, not load-tested:

- **NFR-08 (Resource safety).** IFC parsing is bounded by timeouts, memory caps and vertex limits; uploads are size-limited.

Partially evidenced by structure and tests:

- **NFR-11 (Maintainability).** Layered packages with one-way dependencies, typed ORM models, versioned migrations and a large regression suite.

Implemented; no usability study:

- **NFR-12 (Usability and localisation).** English/Arabic interfaces with RTL and explicit empty, loading and error states.

Architectural goal; not measured:

- **NFR-09 (Scalability).** Correct behaviour with several API workers (advisory locks, LISTEN/NOTIFY); HNSW vector index; fleet-wide processing ceiling.

Not verified:

- **NFR-10 (Performance).** No response-time or throughput target is specified and none was measured.
- **NFR-13 (Accessibility).** Modal focus management is tested; no WCAG conformance audit has been performed.

3.4 System Models and Diagrams

3.4.1 Use case diagrams

The use cases are split into two diagrams to keep them legible. Figure 3.2 covers project execution and collaboration, and Figure 3.3 covers engineering information, AI assistance and administration. Each actor stands for "a user whose effective permissions include the required permission", because roles are configurable.

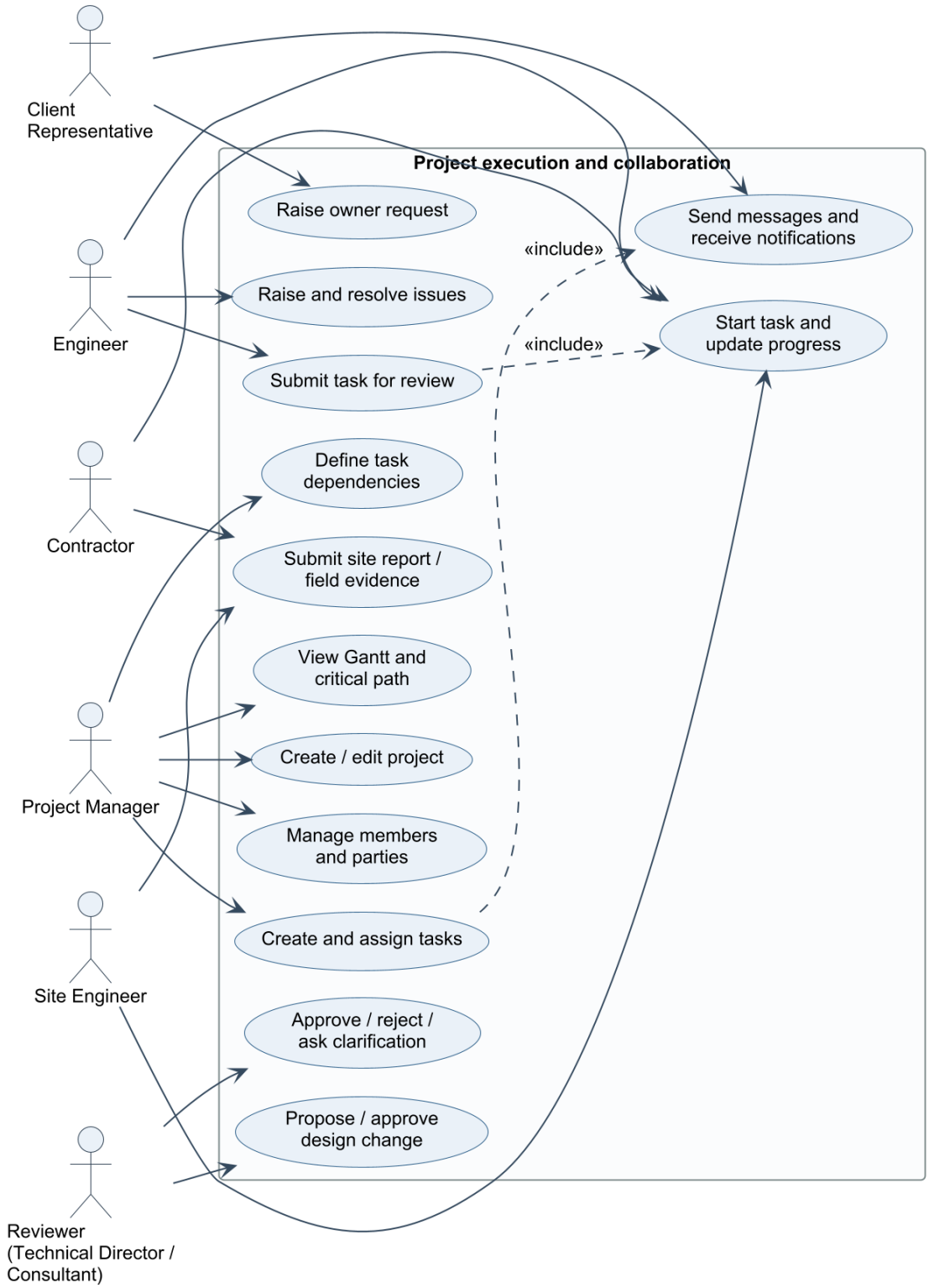


Figure 3.2: Use cases - project execution and collaboration

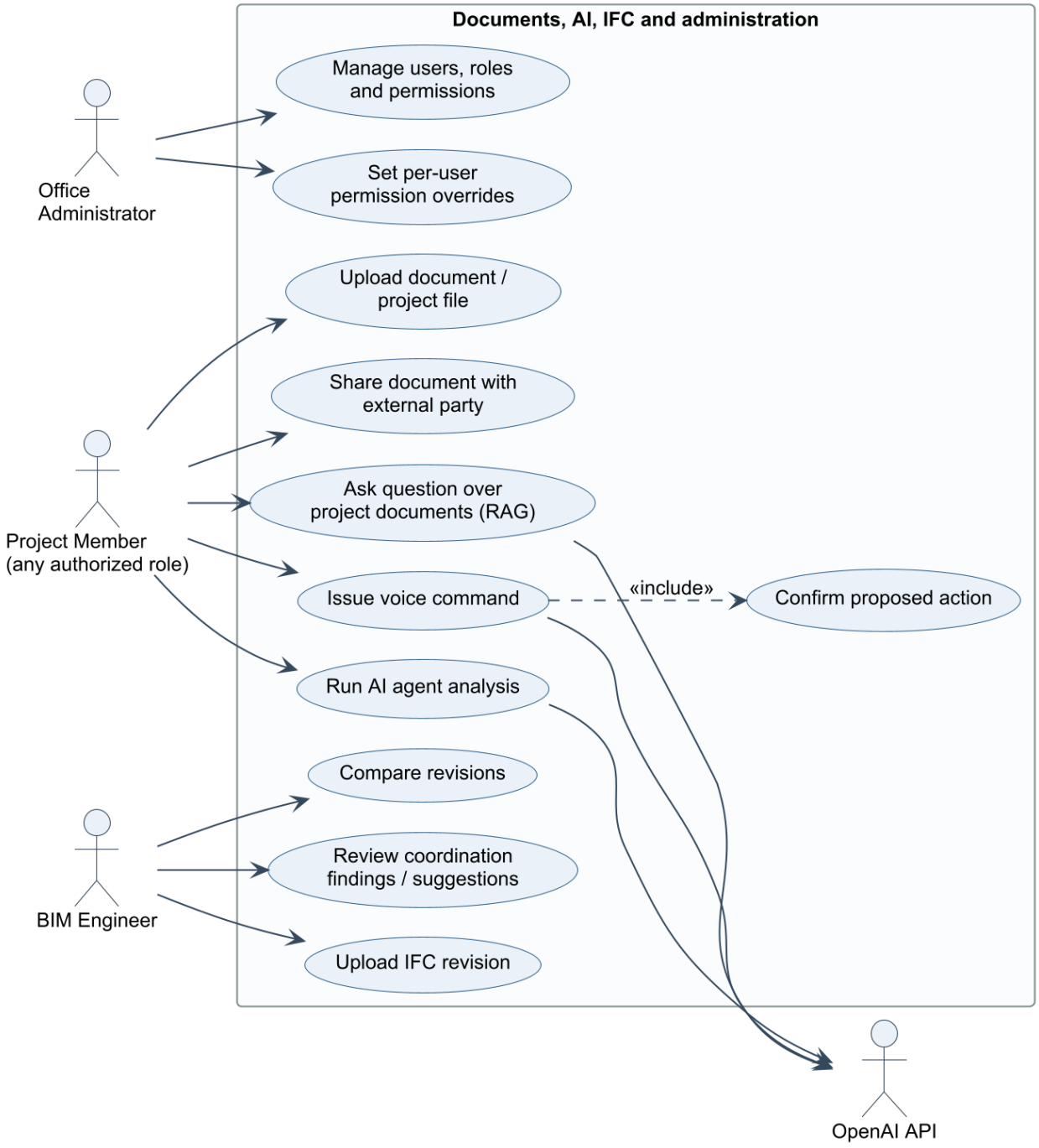


Figure 3.3: Use cases - engineering information, AI assistance and administration

3.4.2 Key use case descriptions

UC-A: Submit a task for review and decide.

- *Actors:* assigned engineer; reviewer holding `task.review`.
- *Precondition:* the task is IN_PROGRESS or REWORK_REQUIRED and requires review.

- *Main flow:* the engineer submits with a completion note; the status becomes UNDER_REVIEW and the reviewer is notified; the reviewer starts the review and approves; the status becomes DONE and an audit entry is written.
- *Alternatives:* the reviewer rejects with required corrections (REWORK_REQUIRED) or requests clarification, which the engineer answers before the decision.
- *Postcondition:* the review history remains in `task_reviews`; progress cannot change while the task is under review or done.

UC-B: Compare model revisions and review their impact.

- *Actors:* BIM engineer (compare permission); project manager (review permission).
- *Precondition:* both versions belong to the same model group and have finished processing.
- *Main flow:* the engineer requests the comparison; the system records element changes, creates impact suggestions and notifies the project manager; the project manager acknowledges or dismisses each suggestion and adjusts the affected tasks.
- *Alternatives:* versions from different groups or still processing are rejected; a model without stable GlobalIds yields a comparison marked LOW confidence.

UC-C: Ask a question about project documents.

- *Actors:* any project member; OpenAI API as secondary actor.
- *Precondition:* RAG is enabled and the project has indexed sources the user may read.
- *Main flow:* the system determines readable sources, embeds the question, retrieves the top passages from those sources only, and returns the model's answer with citations built from the retrieved records.
- *Alternatives:* with no readable sources, a not-found answer is returned without any model call; if the model reports insufficient context, the user sees that the information was not found.

3.4.3 Activity diagram - task execution and review

Figure 3.4 models UC-A including the blocker loop and the two completion paths. Its guard conditions are the checks performed by the task endpoints.

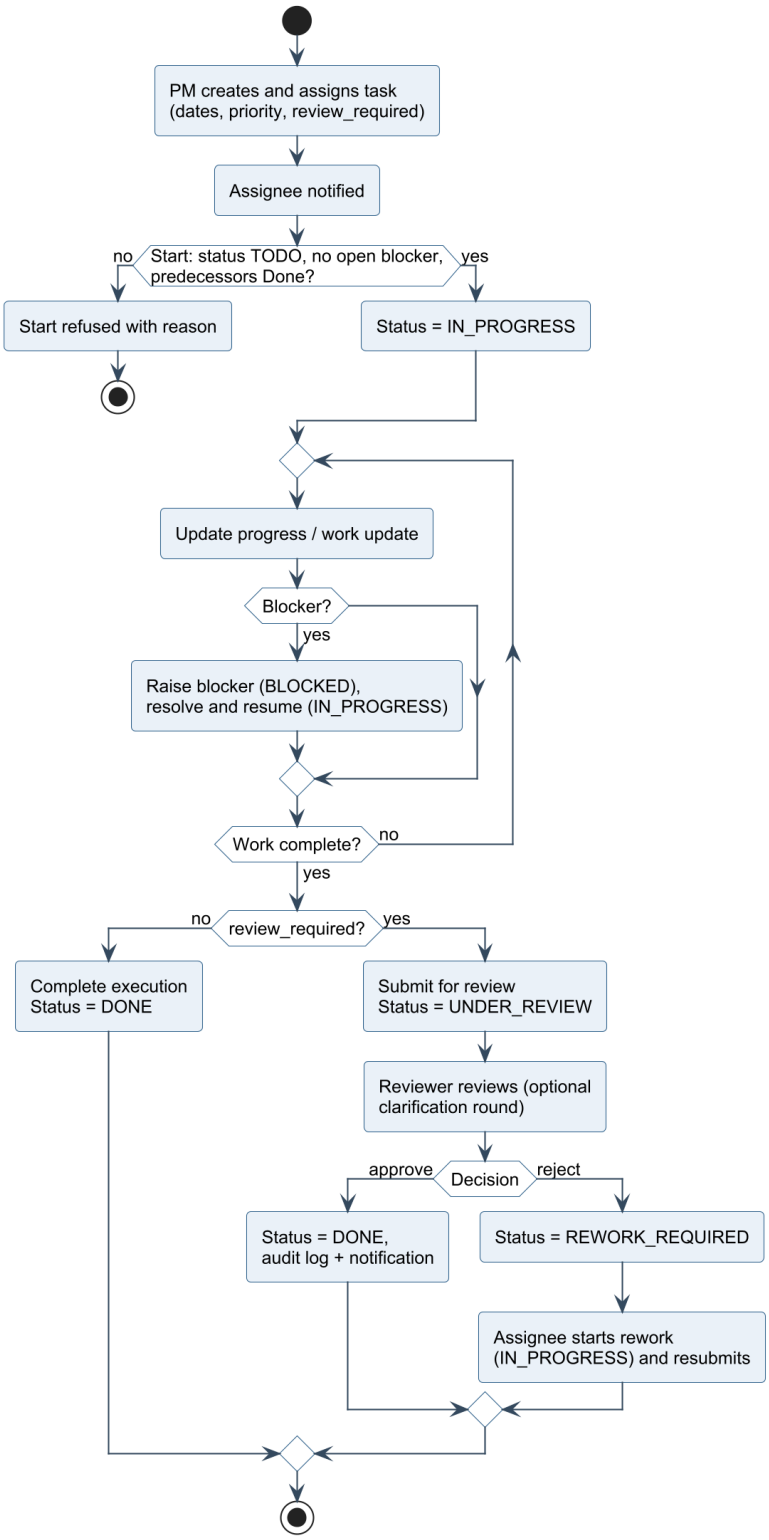


Figure 3.4: Activity diagram - task execution and review

3.4.4 State machine diagrams

Three entities have lifecycles whose transitions are enforced in code. Tasks (Figure 3.5) follow the transitions of the task endpoints. IFC model versions (Figure 3.6) and ingested files (Figure 3.7) are governed by explicit transition tables that reject any unlisted change, which prevents a late or duplicated worker from overwriting a terminal state.

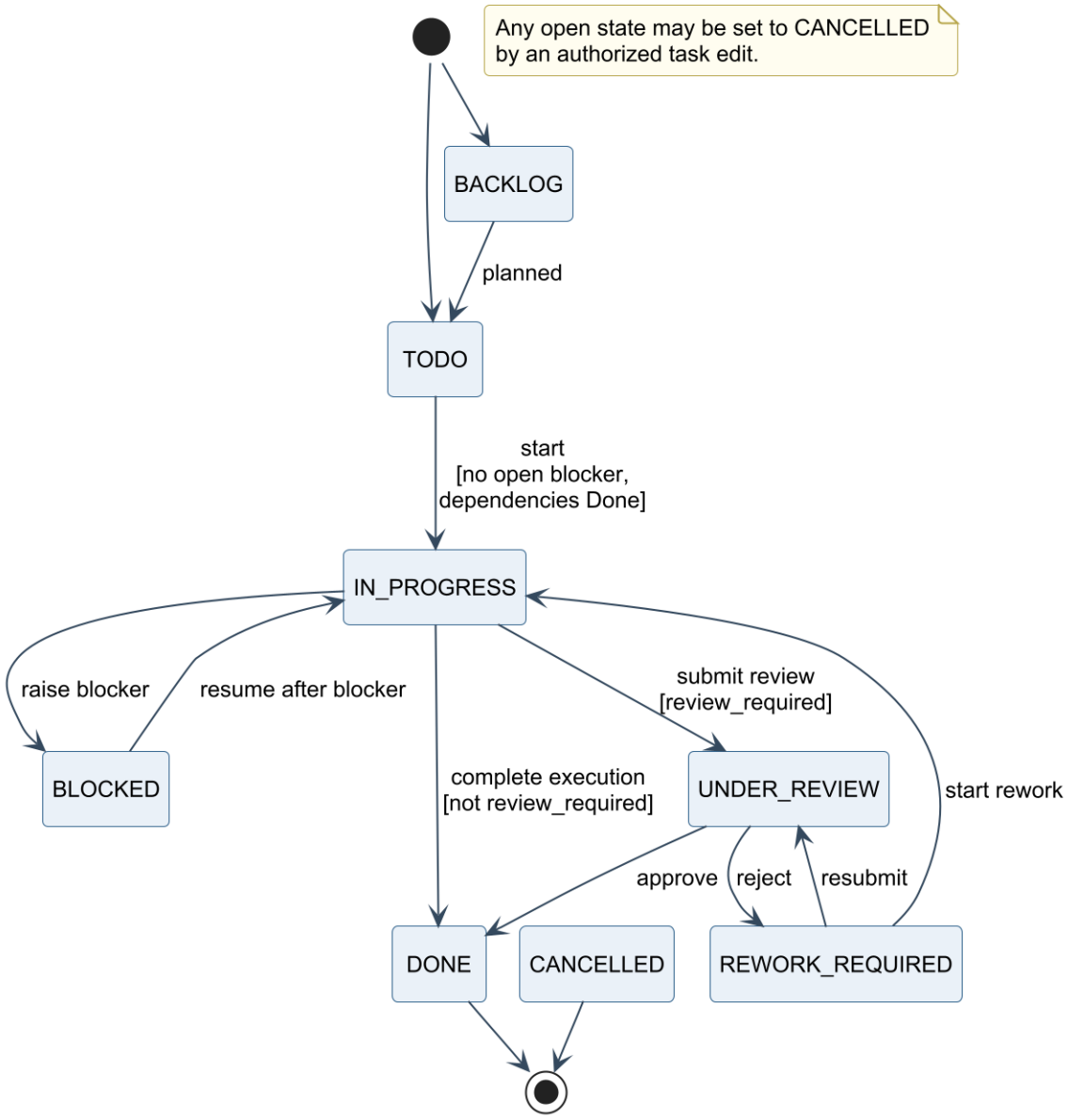


Figure 3.5: State machine - task status

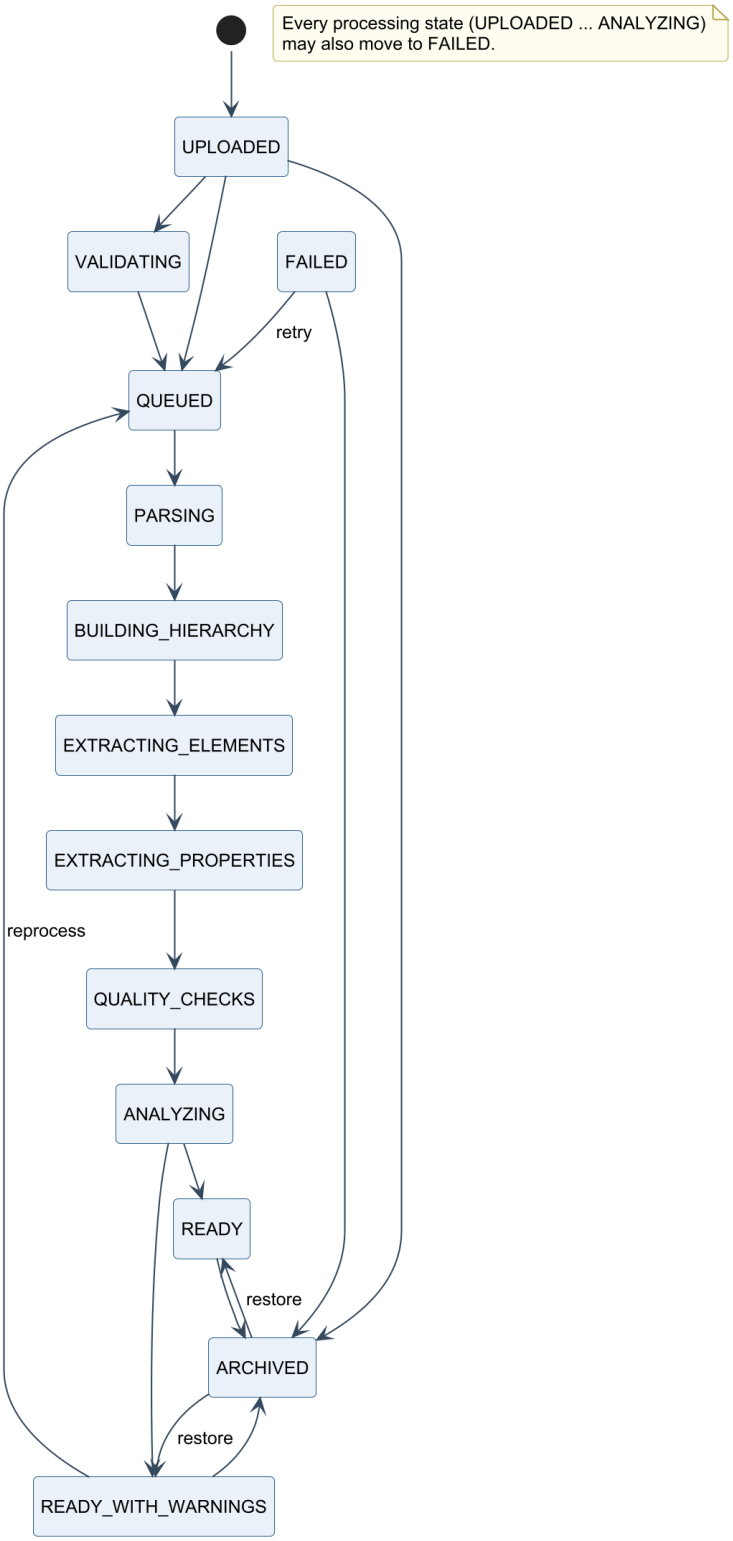


Figure 3.6: State machine - IFC model version processing status

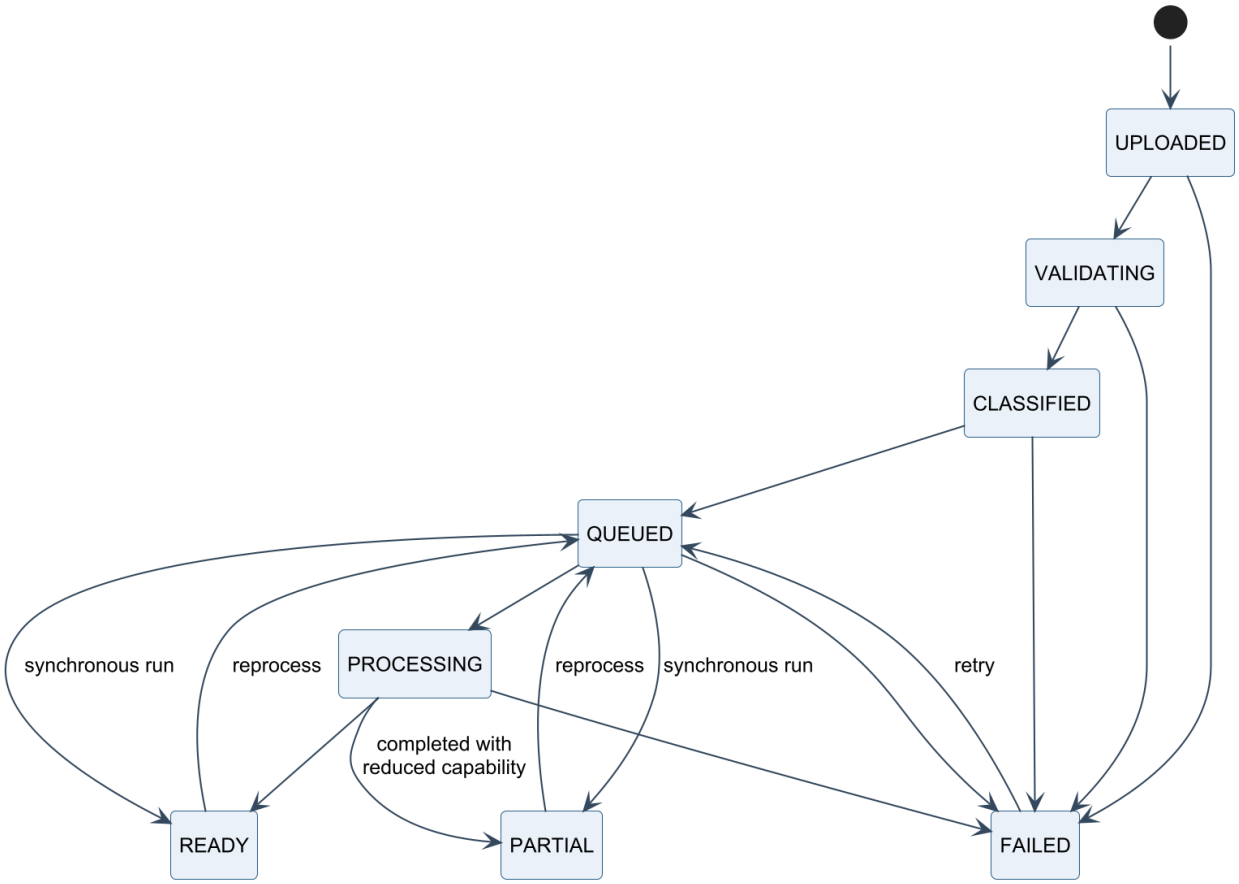


Figure 3.7: State machine - ingested file status (PARTIAL is a usable outcome with reduced capability)

Chapter 4: System Design and Architecture

This chapter reconstructs the architecture from the repository: tiers and components, deployment, the data model, interfaces, the main data flows, end-to-end scenarios and the design patterns the code actually uses.

4.1 High-Level Architecture

Constructs is a three-tier client-server system (Figure 4.1). The web and mobile clients call one FastAPI backend over HTTPS with JSON and bearer tokens. Inside the backend, the API layer handles routing, validation and status codes; the service layer holds all domain logic, policies and integrations; and the data layer consists of typed SQLAlchemy models. A separate worker process runs the same code against the same database and executes queued jobs, so heavy work never runs on the request path.

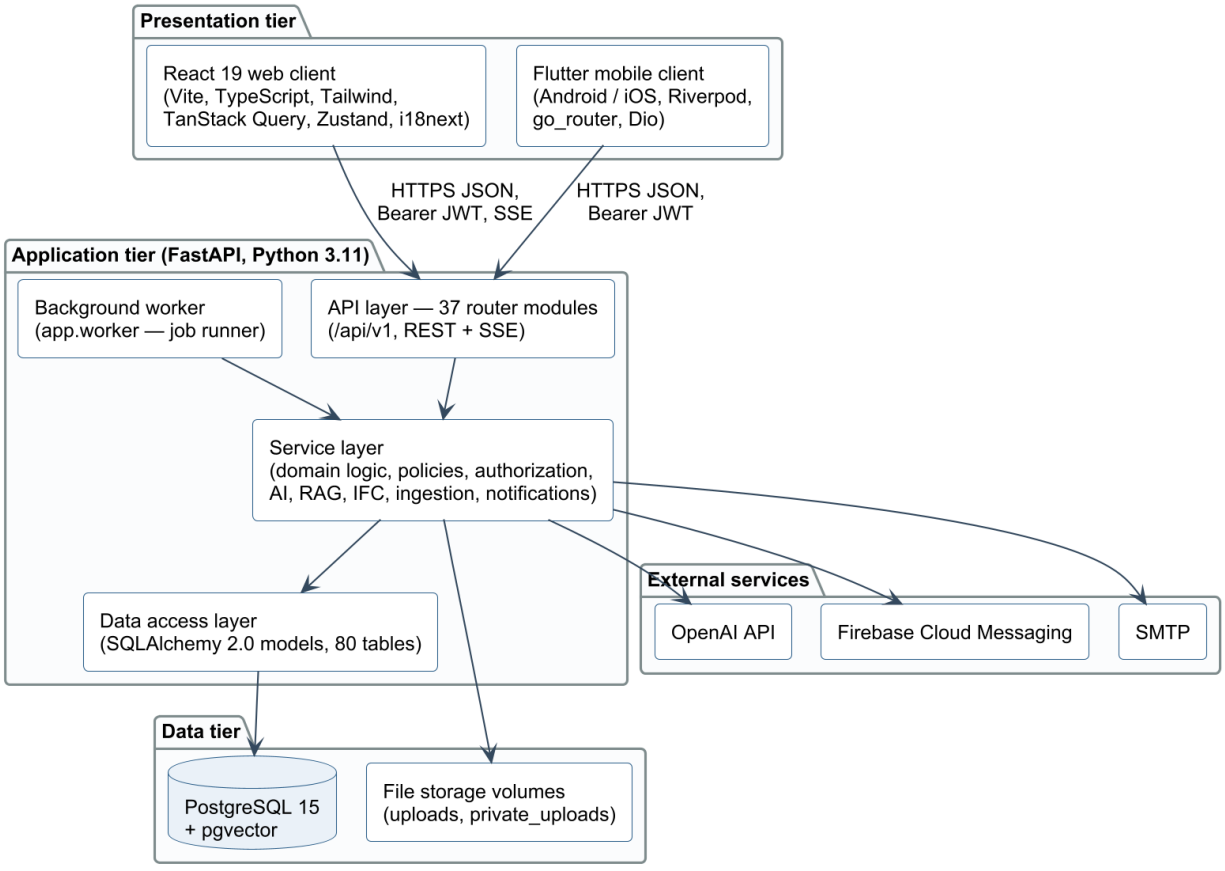


Figure 4.1: High-level architecture

One deliberate decision shapes the whole design: PostgreSQL is used not only for data but also as the job queue, the lock service, the realtime message bus and the vector index. This keeps the operation of the system to a single stateful service, which suits a small team. The cost is that throughput is bounded by the database; the code mitigates this with `SKIP LOCKED` claiming, advisory locks and an HNSW index, and the storage interfaces (`VectorStore`, `PrivateStorage`) leave room to move to external services later.

Figure 4.2 shows the main backend components. API modules depend on domain services; domain services use three cross-cutting services - authorization, the job queue and audit/domain events. Authorization is called on every protected path, including voice, AI tools, agents and RAG; drawing all those edges would make the diagram unreadable, so they are summarised in a note.

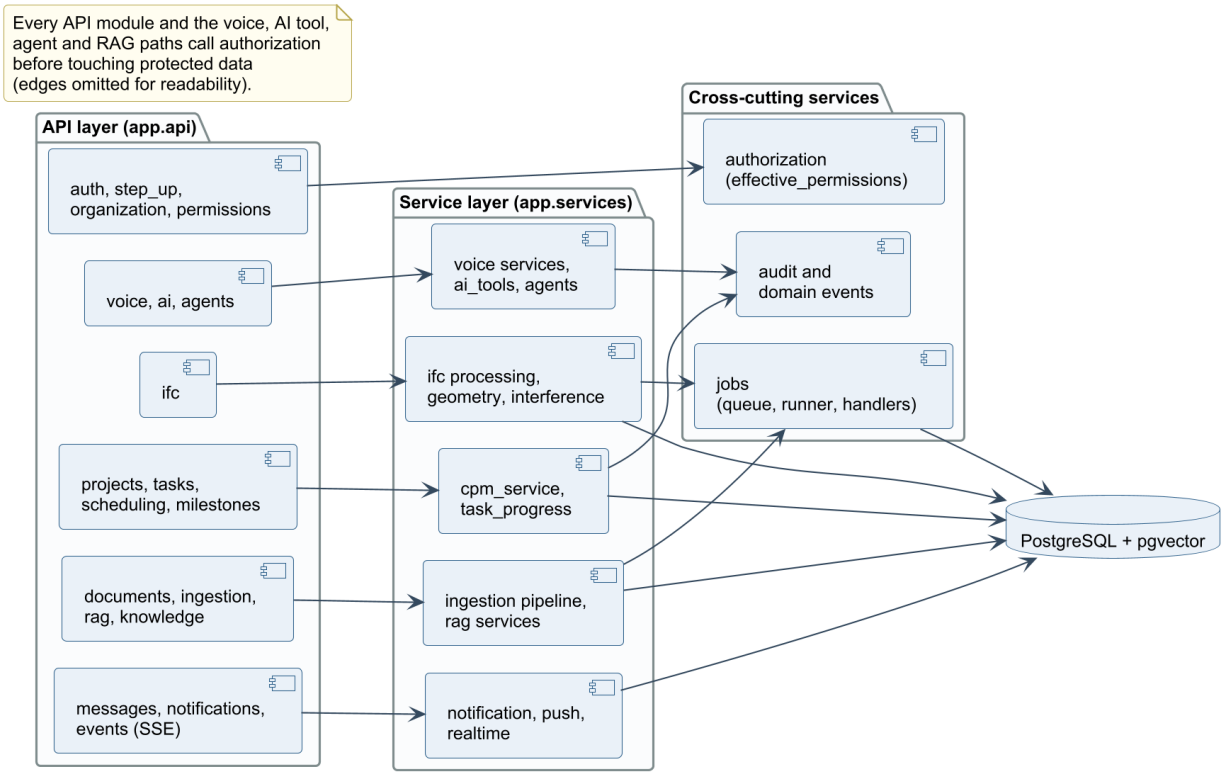


Figure 4.2: Backend component diagram

The deployment in `docker-compose.yml` is shown in Figure 4.3. Four services start by default - database, API, worker and web frontend; a fifth, `flutter-tools`, runs only under the `mobile-tools` profile for repeatable mobile builds. Named volumes hold the database and uploaded files, and the Firebase credential is mounted read-only from a git-ignored directory. The single container start sequence - wait for the database, migrate, seed roles, optional demo seed, serve - is guarded by a test so that every environment starts the same way.

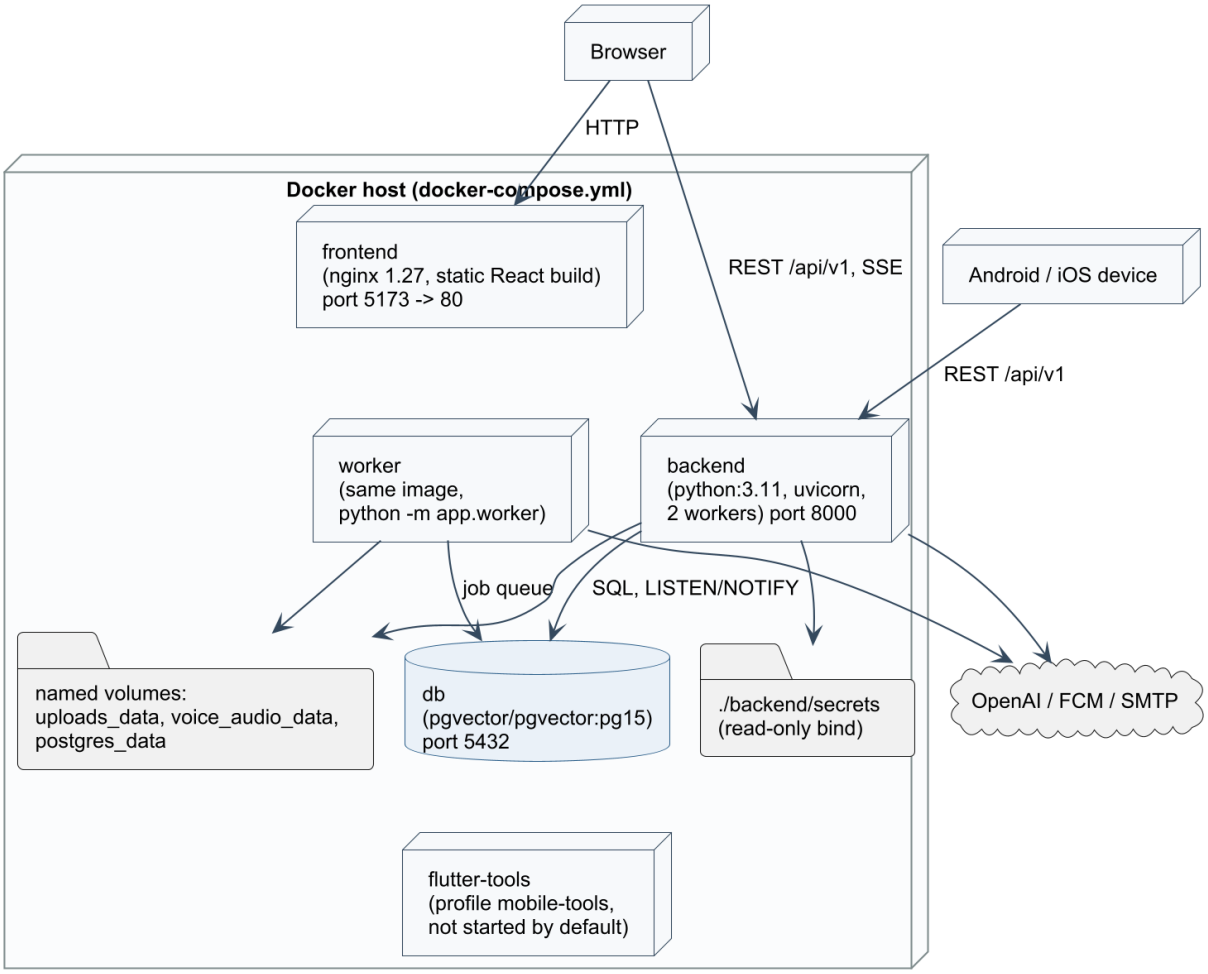


Figure 4.3: Deployment diagram (Docker Compose)

4.2 Detailed Design, Data Design, and Interface Design

4.2.1 Backend structure

The backend packages have one-way dependencies (API, then services, then models). `app/api` contains only the HTTP surface; `app/schemas` holds Pydantic request and response contracts; `app/services` contains the domain logic and integrations; `app/models` declares the ORM entities; `app/core` holds configuration, security primitives, dependencies and the permission catalogue; `app/ai` contains the model-facing parts of the voice pipeline; and `app/db` contains session management, the administrator bootstrap and the demo seed.

4.2.2 Data design

The database declares 80 tables with consistent conventions: UUID primary keys, creation and update timestamps from mixins, JSONB for open-ended payloads such as IFC properties and AI

evidence, check constraints for closed vocabularies, and partial unique indexes for rules such as "unique per organisation". The four entity-relationship diagrams below cover the core of each domain and abbreviate attributes to those that carry relationships or business meaning; the complete schema is defined by the migrations.

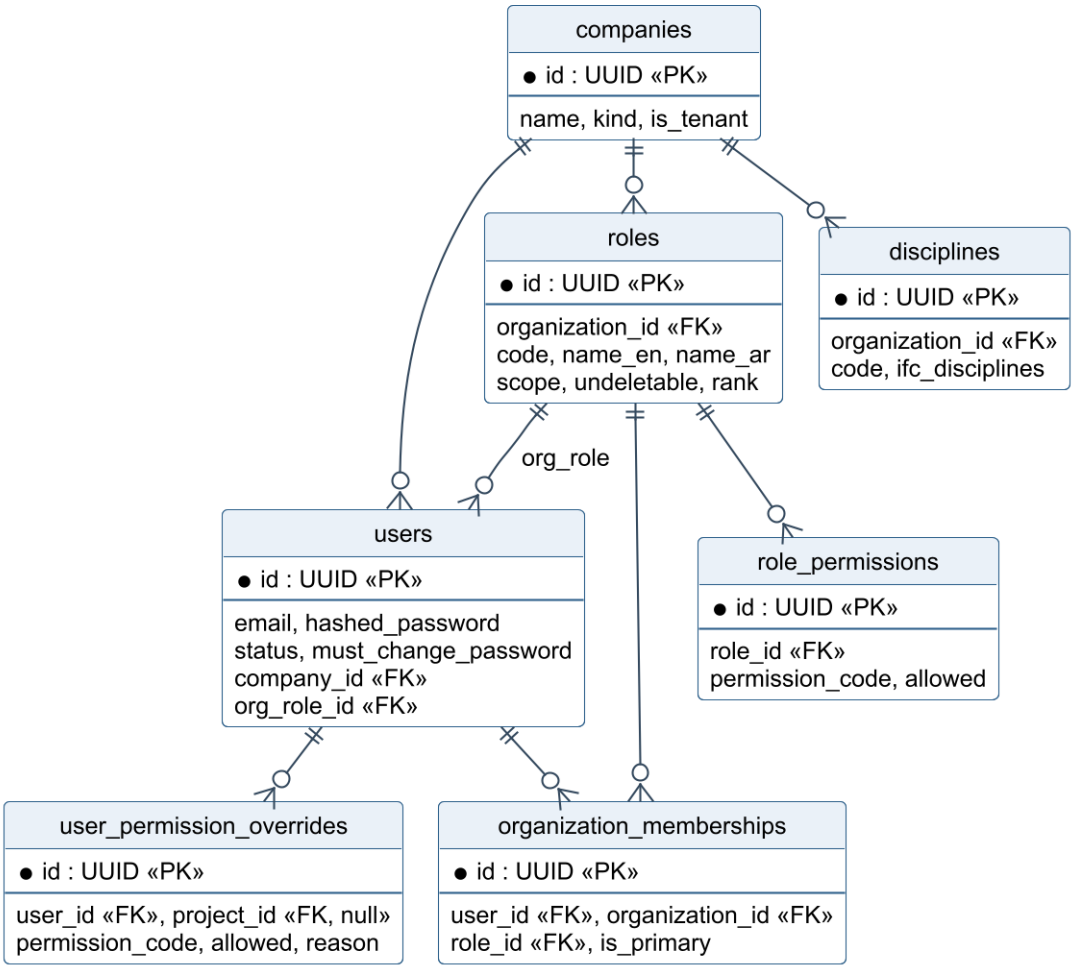


Figure 4.4: ER diagram - identity, organisation and RBAC

In Figure 4.4, every user has exactly one office role (`users.org_role_id` is mandatory and is the only source of office authority). `role_permissions` stores configurable grants and `user_permission_overrides` stores explicit allow or deny decisions, optionally limited to one project.

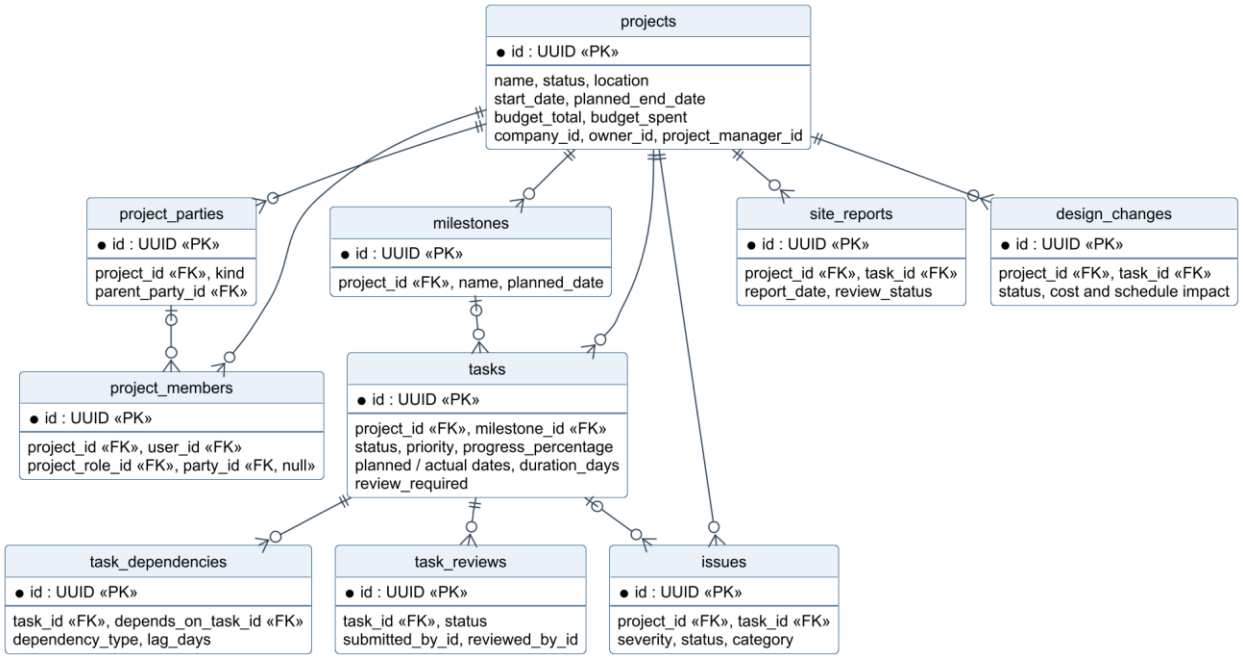


Figure 4.5: ER diagram - projects, parties, tasks and field records

In Figure 4.5, `project_members.party_id` is empty for office staff and points to a `project_parties` row for an external participant; this single column is how the authorization service decides whether a person is external on a given project. `task_dependencies` stores the precedence network used for CPM.

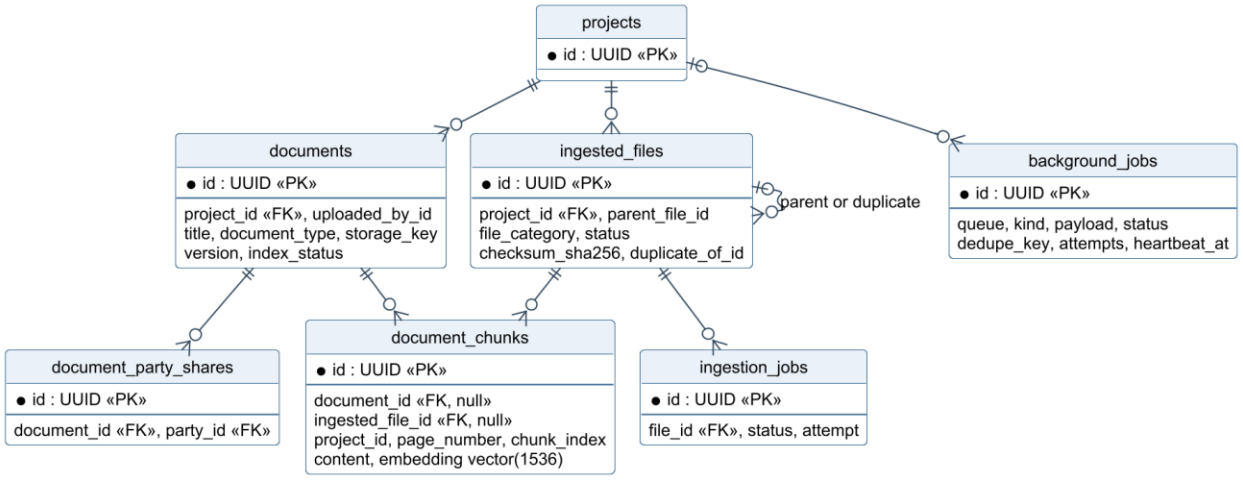


Figure 4.6: ER diagram - documents, ingestion, retrieval chunks and jobs

Figure 4.6 shows that a retrieval chunk belongs either to a library document or to an ingested file and carries the project id, so retrieval can filter by project and by readable source in one indexed query. The embedding is a `vector(1536)` column with an HNSW cosine index.

`document_party_shares` implements deny-by-default external access: without a row, a contractor cannot see a document.

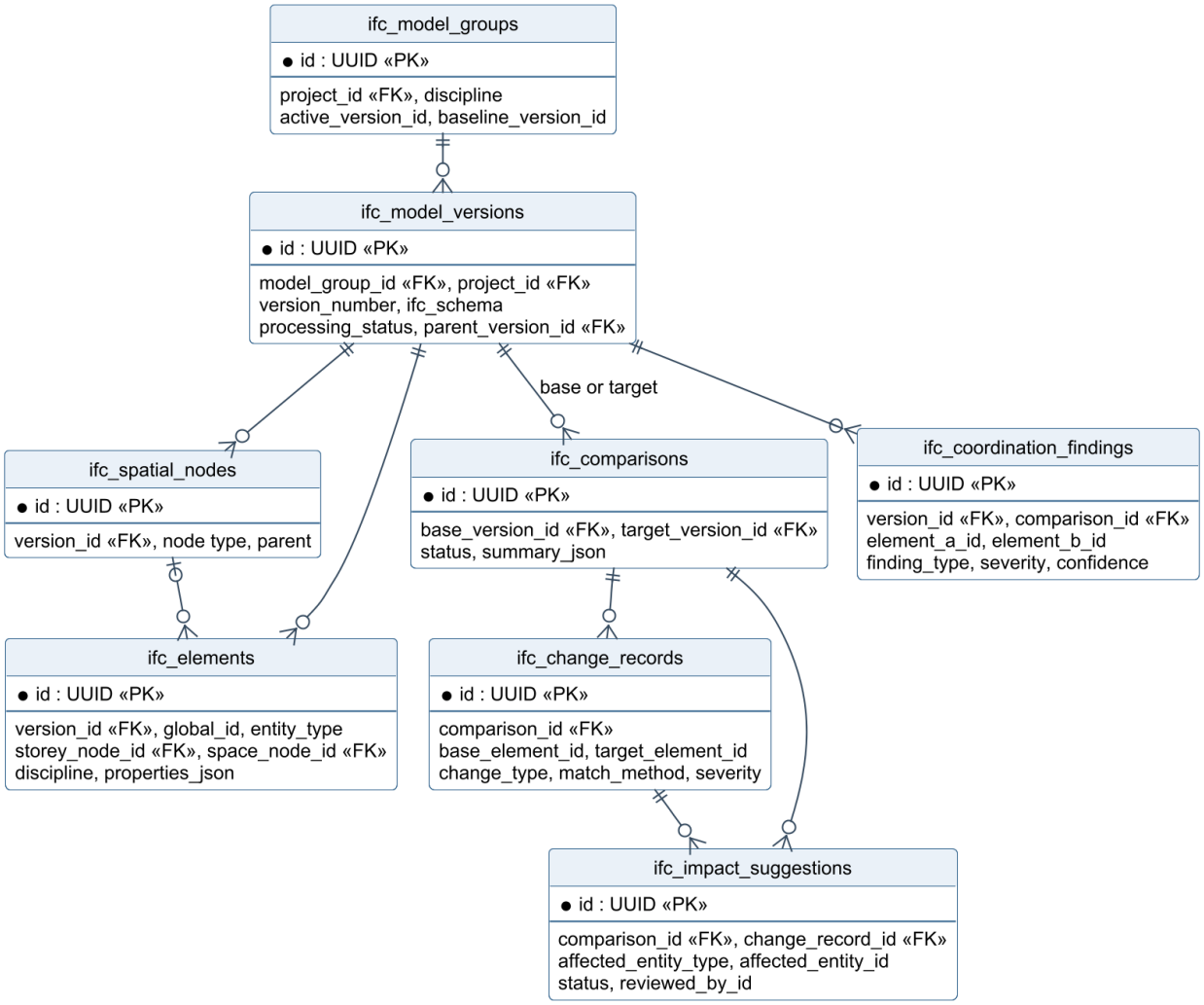


Figure 4.7: ER diagram - IFC model groups, versions, elements and analysis results

In Figure 4.7, a model group (for example the structural model) holds numbered versions. Elements keep their IFC GlobalId, which allows two versions to be compared element by element. Comparison output is normalised into change records, impact suggestions and coordination findings, each reviewable individually. `ifc_entity_links` (not drawn, to keep the diagram readable) links an element to a task, issue, milestone, document, site report, field submission or media asset; these links drive the high-confidence impact suggestions.

4.2.3 Interface design

The REST API is mounted under `/api/v1` and exposes 337 route handlers in 38 router modules. Login uses the OAuth2 password form; every other call carries a bearer access token. The largest

groups are IFC (53 handlers: model groups, versions, hierarchy, elements, geometry, comparisons, impacts, links, suggestions, findings), tasks (31), projects (20), collaboration (17), messages (16), users and site reports (14 each) and voice (13). Live updates use `POST /events/ticket` followed by `GET /events/stream`, because a browser `EventSource` cannot send an authorization header.

The web client is organised by feature (`src/features/<domain>`), with 23 feature folders, TanStack Query for server state, Zustand stores and lazily loaded routes under one URL scheme per project, so navigation does not depend on role names. Route guards hide what a user cannot do, but they are presentation only: the server re-checks every call. The mobile client has 17 feature folders - tasks, site reports, field evidence, voice, IFC list, messages, notifications and others - and uses Riverpod, go_router and Dio, keeping tokens in secure storage.

4.3 Data Flow and Process Models

Every protected request passes through permission resolution (Figure 4.8). Overrides are applied before the ceilings, so even a mistaken grant to an external participant is removed before the check is made.

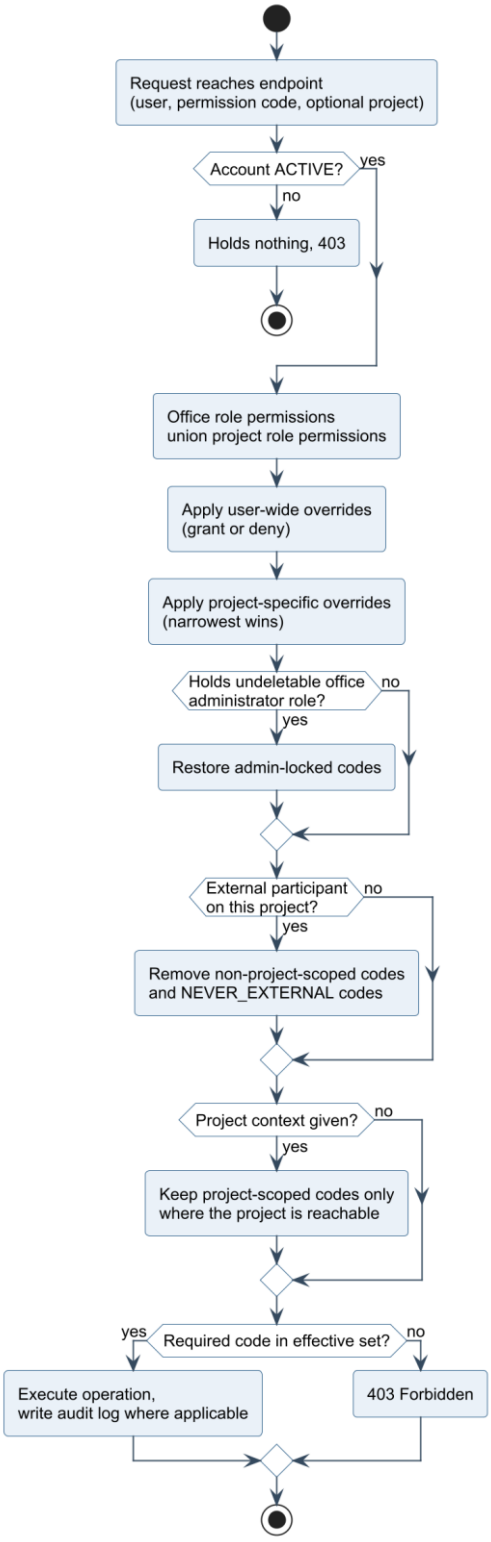


Figure 4.8: Activity diagram - effective permission resolution

Figure 4.9 shows login and transparent token refresh. Because refresh tokens are single-use, the web client shares one in-flight refresh among concurrent requests that receive a 401 response; otherwise the second request would present an already revoked token.

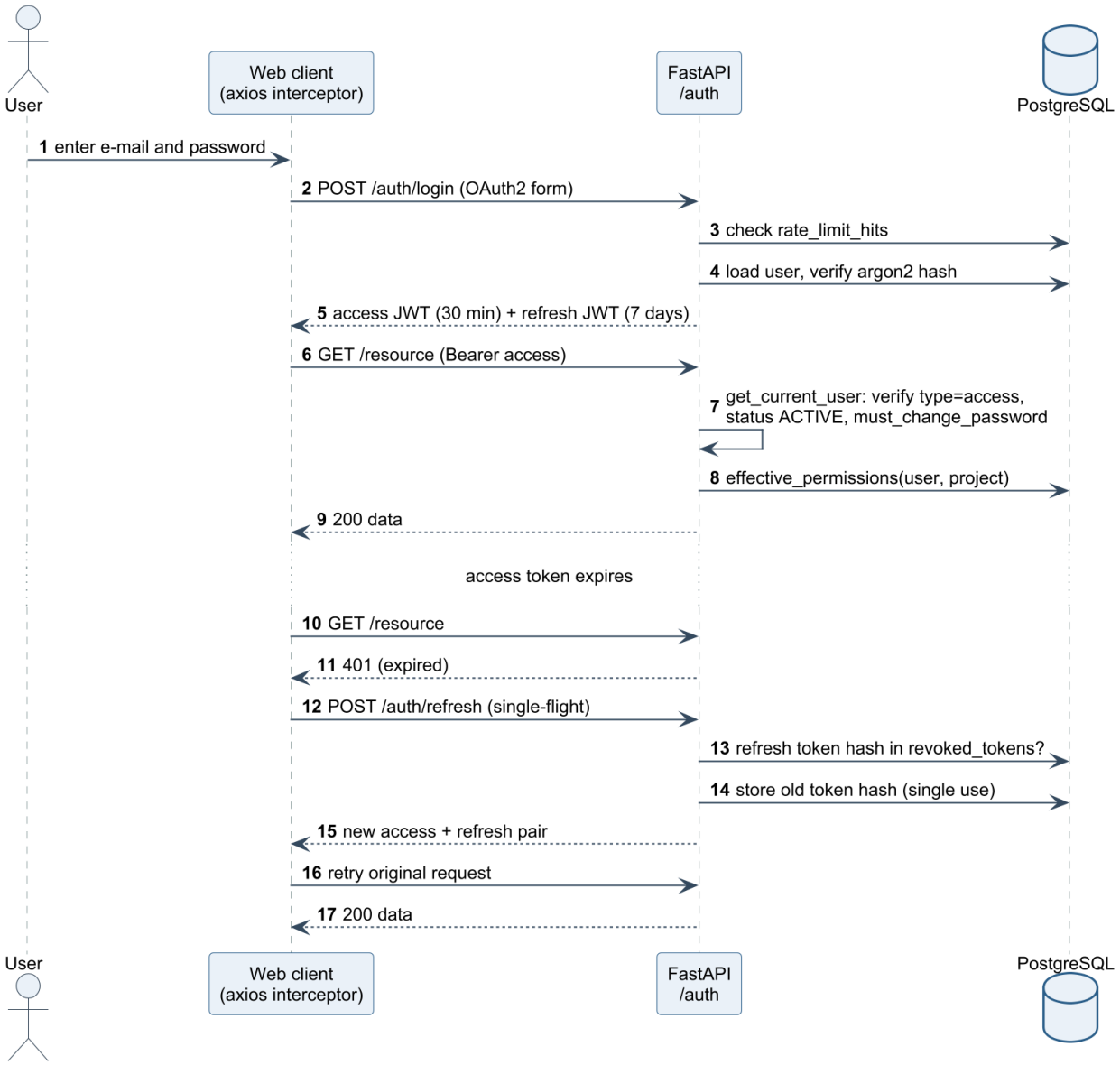


Figure 4.9: Sequence diagram - login, authorised request and token refresh

Figure 4.10 shows the background job lifecycle. A job row is inserted in the same transaction as the upload or event that caused it, so a rolled-back request leaves no orphan work; workers claim jobs with `FOR UPDATE SKIP LOCKED`, and a transaction-scoped advisory lock enforces a fleet-wide concurrency ceiling.

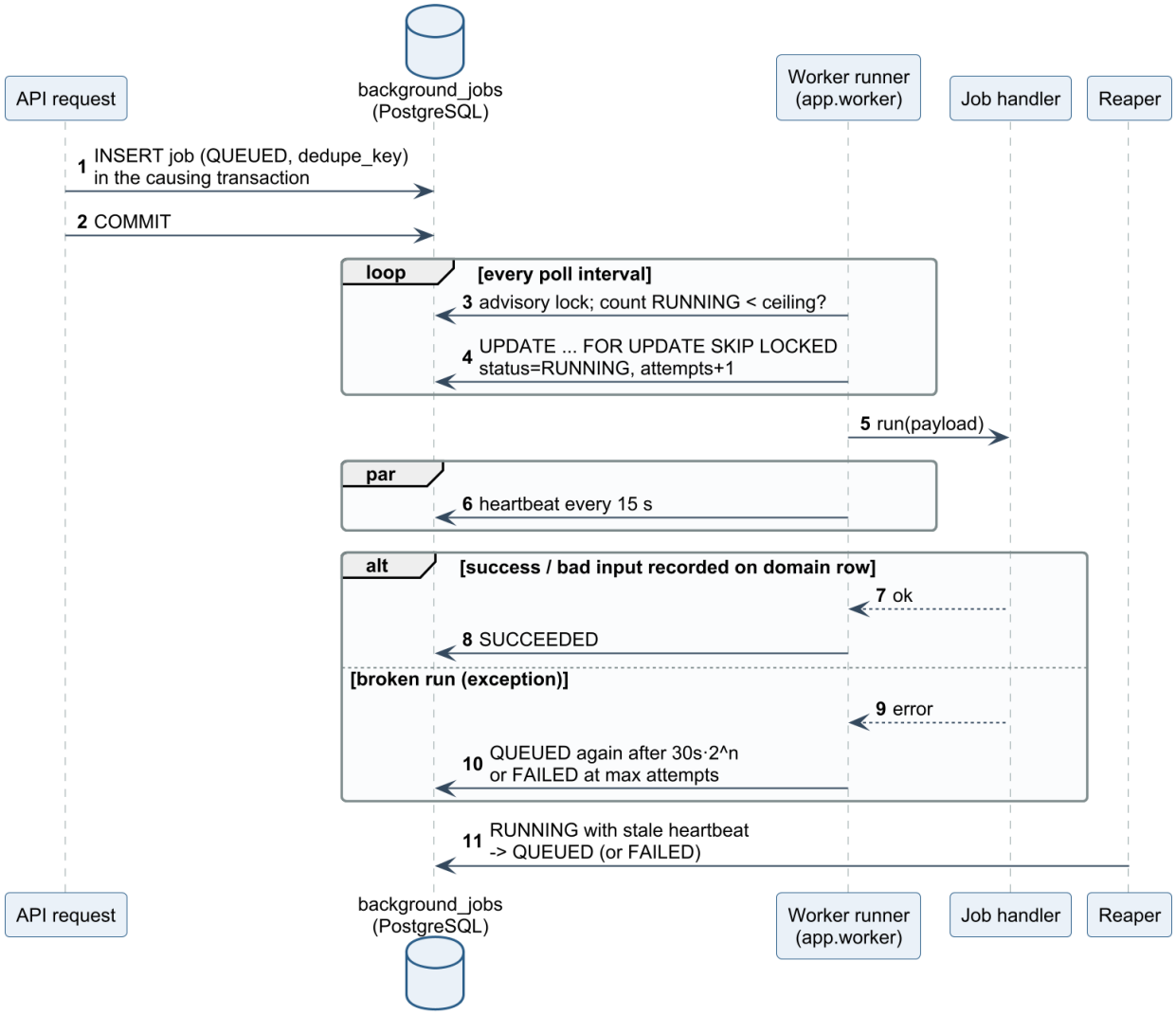


Figure 4.10: Sequence diagram - durable background job processing

Figure 4.11 is a level-1 data flow diagram of file ingestion and document question answering. The only data that leaves the platform is text sent to the OpenAI API for embedding and answering, and only when RAG is enabled.

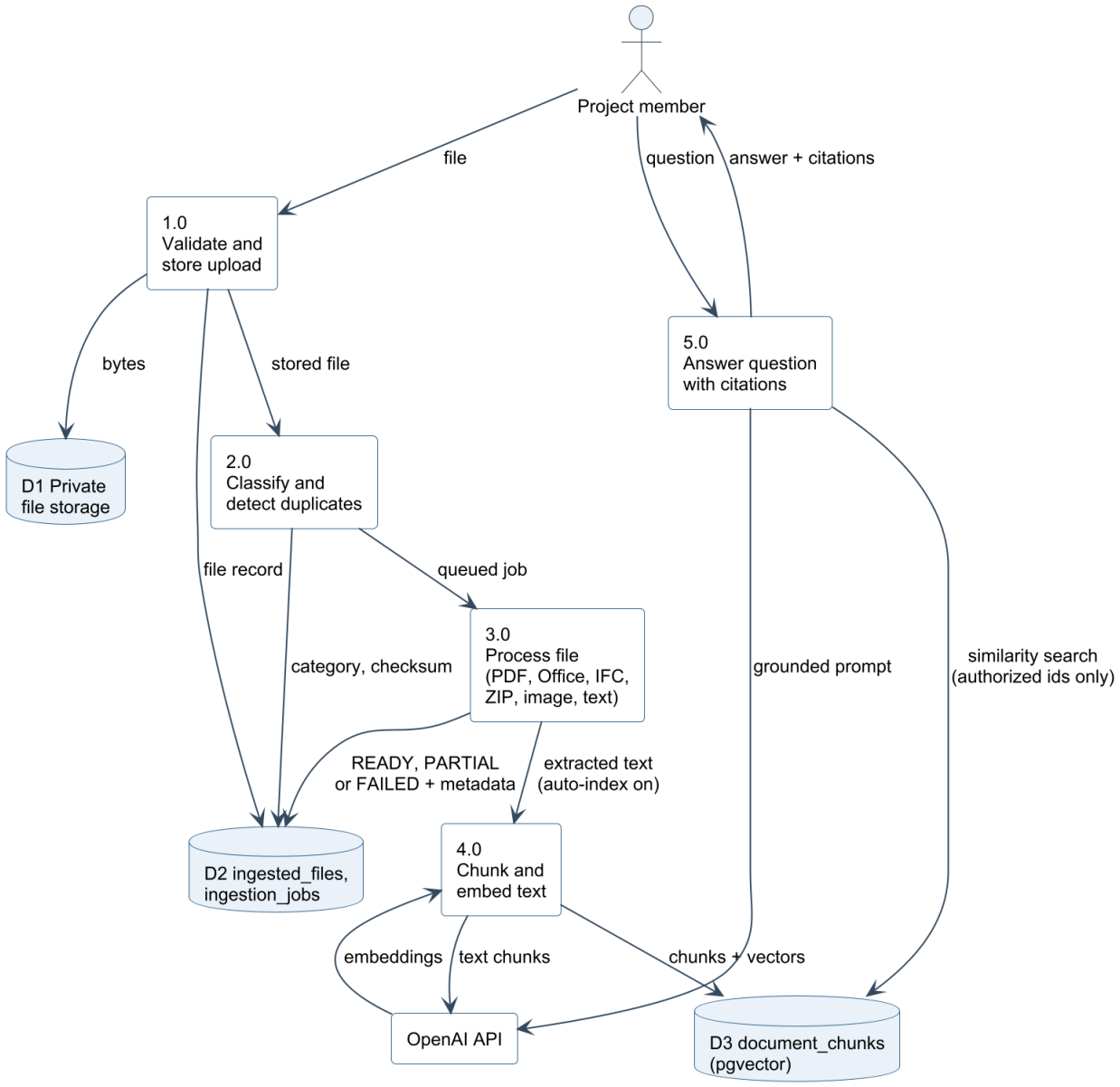


Figure 4.11: Level-1 data flow diagram - file ingestion and document question answering

Figure 4.12 follows an IFC upload from the request into the worker. The upload transaction stores the file and queues a job; parsing happens later in an isolated subprocess, so a malformed or oversized model cannot exhaust the API process. Figure 4.13 then shows what happens when two processed versions are compared.

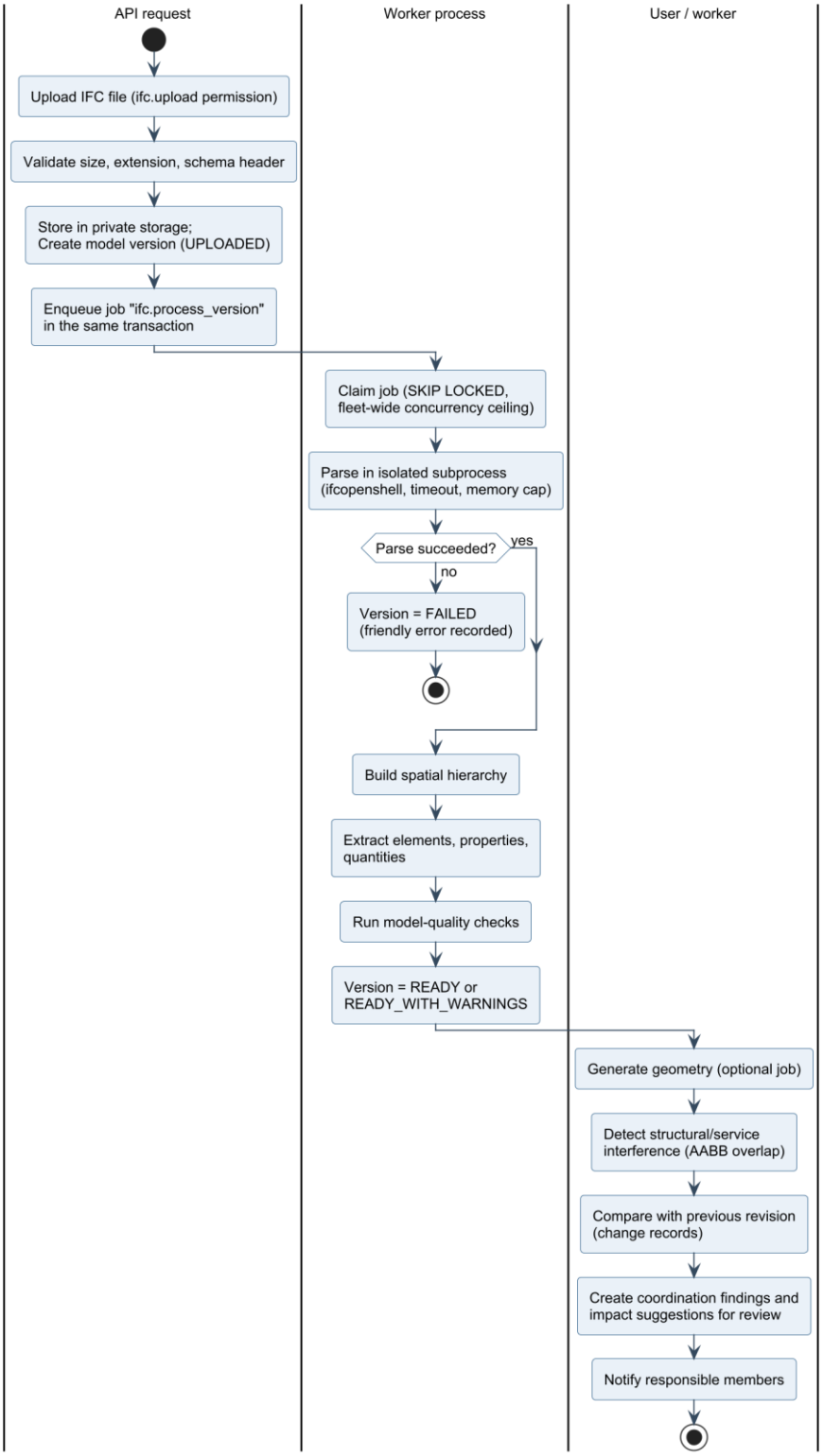


Figure 4.12: Activity diagram - IFC upload, processing and coordination

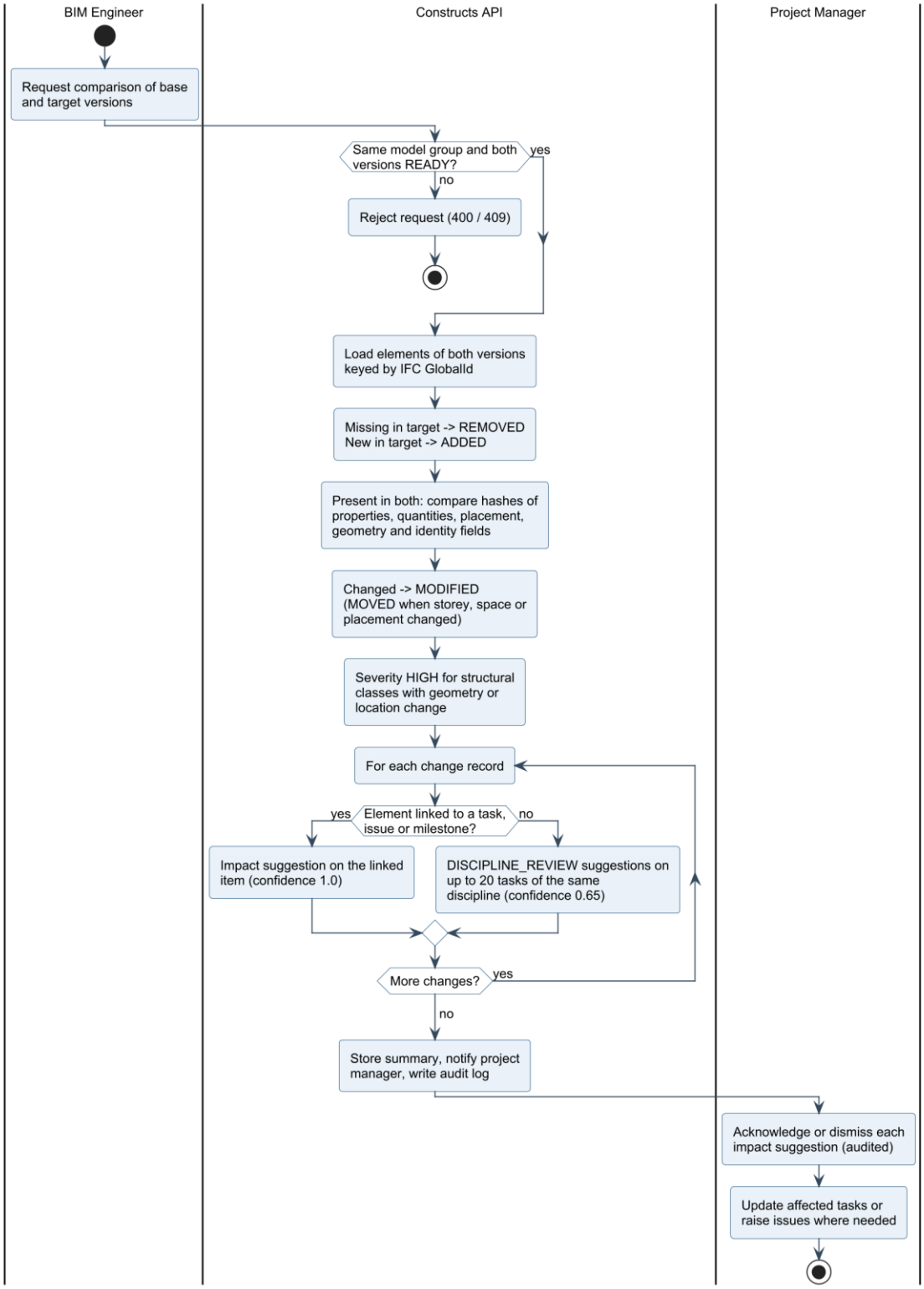


Figure 4.13: Activity diagram - revision comparison and impact suggestions

4.4 End-to-End Scenarios

The following scenarios show how modules cooperate along paths that exist in the code. They are described from the implementation and its integration tests; they have not been validated with real users.

4.4.1 Scenario 1 - from a contract PDF to a cited answer

1. A document controller uploads a contract PDF; the ingestion pipeline validates it, computes its checksum, classifies it and queues processing (FR-13).
2. The worker extracts the text layer; because automatic indexing is on, an indexing job splits the text into overlapping chunks, embeds them and stores them with page numbers (FR-20).
3. An engineer asks what retention percentage applies. The knowledge router recognises a document question (FR-21); authorization computes which documents the engineer may read; retrieval searches only those chunks.
4. The model answers from the numbered passages; Constructs attaches citations built from the retrieved chunk records, so each citation points to a real document and page (Figure 4.14).

4.4.2 Scenario 2 - from a model revision to affected work

1. A BIM engineer uploads revision B of the structural model; the worker parses it and the version becomes READY (FR-15).
2. The engineer compares revision A with B. Elements are matched by GlobalId; a column whose placement changed is recorded as MOVED with HIGH severity (FR-17).
3. The column is linked to a task, so an impact suggestion with confidence 1.0 is created on that task; changed elements without links produce lower-confidence discipline suggestions. The project manager is notified.
4. The project manager acknowledges the suggestion and reschedules the task; a coordination finding on a duct crossing a beam is turned into an issue linked back to the element (FR-18).

4.4.3 Scenario 3 - from a voice report on site to an updated plan

1. A site engineer records, in Arabic, that the ground-floor slab formwork is seventy percent complete (FR-25).
2. The audio is stored privately, transcribed and analysed; the router identifies a progress update rather than a question, and the matcher resolves the task (Figure 4.15).

3. The engineer reviews the proposed draft and confirms it within ten minutes; the action service re-checks permissions and updates the task, recording an audit entry and the before and after state.
4. The project manager receives a notification and the open web client refreshes through a realtime hint (Figure 4.16).

The three sequence diagrams referenced by the scenarios are wide and are therefore placed on landscape pages. Figure 4.14 shows retrieval-augmented answering: authorization is resolved first and passed into retrieval as two explicit id sets, which have no default of "all documents".

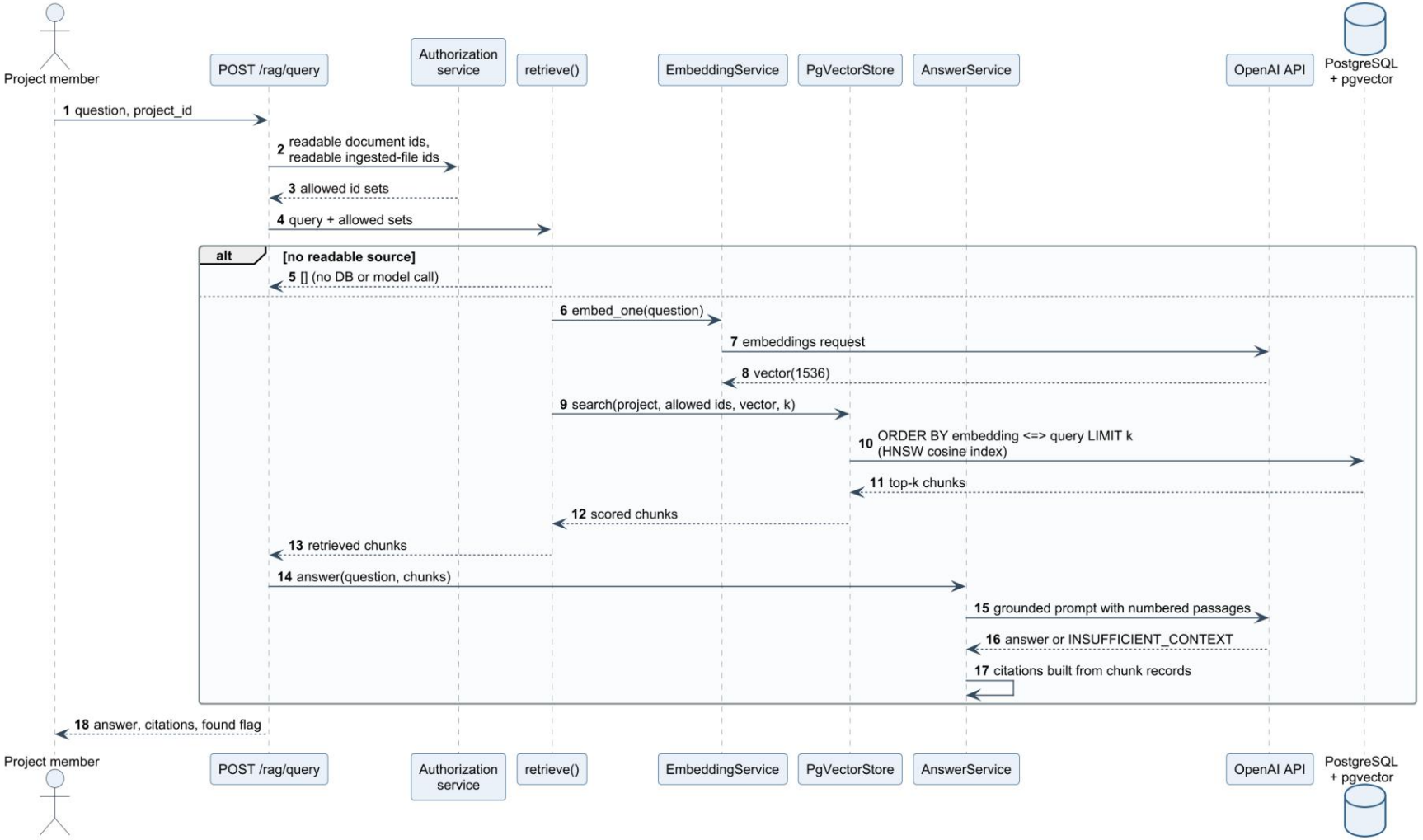


Figure 4.14: Sequence diagram - retrieval-augmented document question answering

Figure 4.15 shows a voice command. The model determines intent and entities; deterministic routing decides between a question and a change; a change is executed only after confirmation and a fresh permission check.

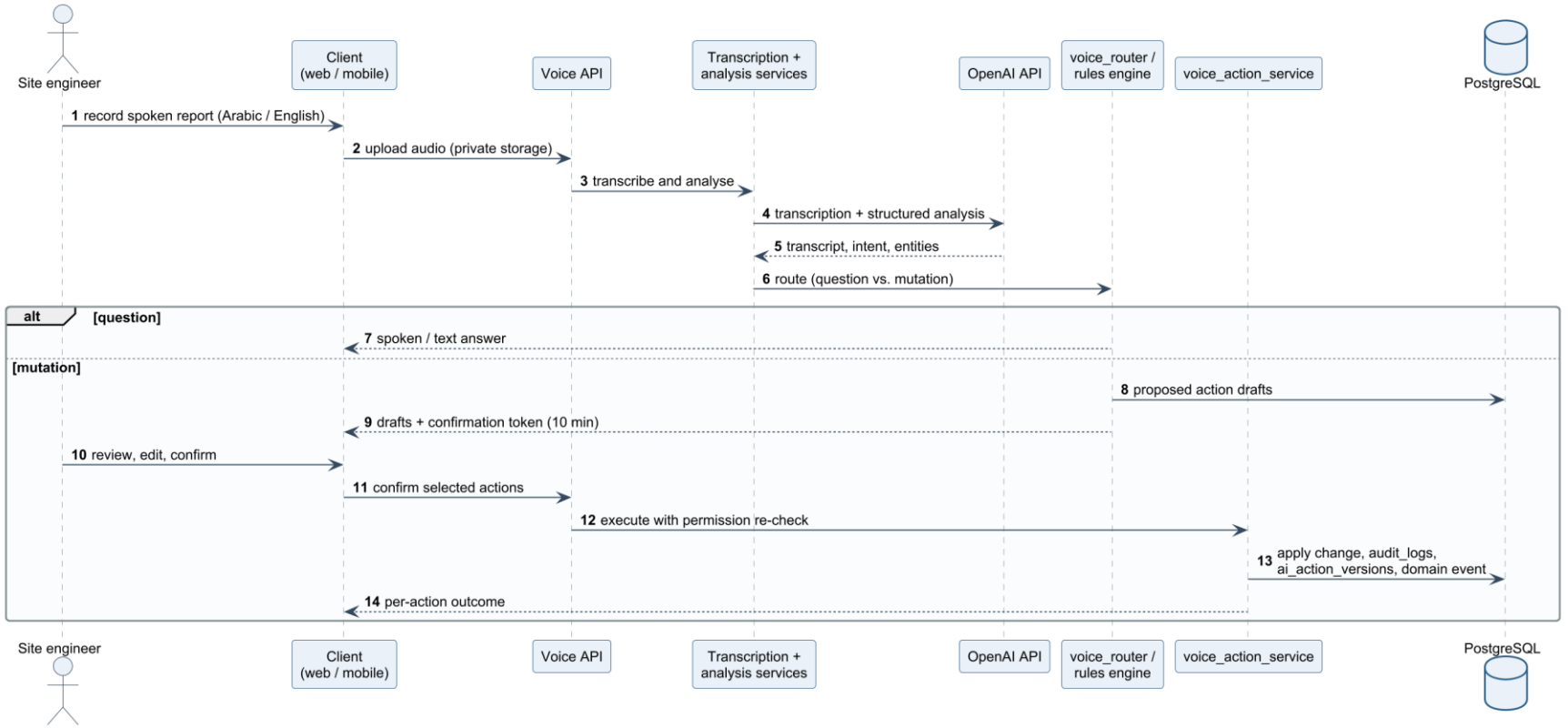


Figure 4.15: Sequence diagram - voice command to confirmed action

Figure 4.16 shows notification fan-out. The notification row and the realtime event are written in the caller's transaction; PostgreSQL delivers NOTIFY messages only after commit, and events carry only identifiers so that clients re-fetch data through authorised REST calls.

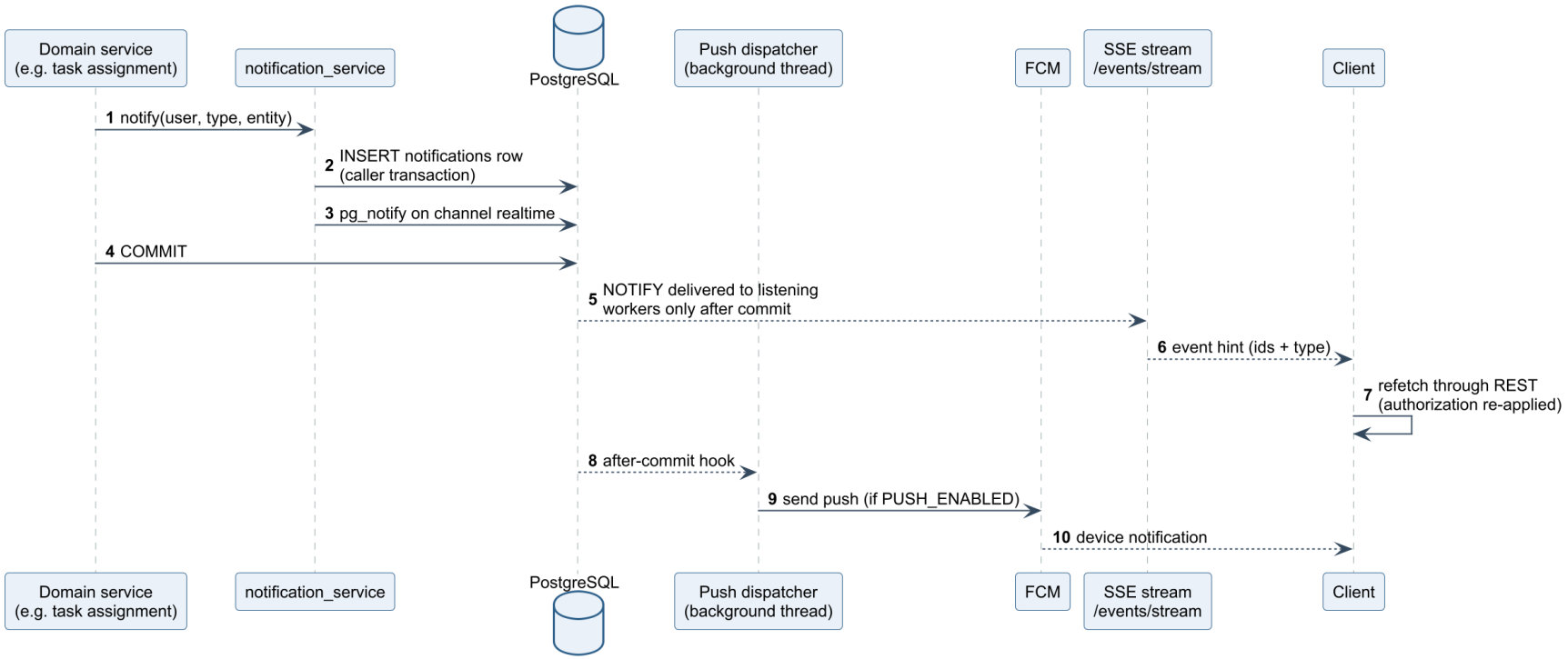


Figure 4.16: Sequence diagram - notification, realtime and push fan-out

4.5 Design Patterns and Justifications

Patterns are named only where the code clearly matches their intent.

- **Layered architecture** - `app/api` to `app/services` to `app/models`; the same services are reused by REST, voice, the MCP tool surface and agents.
- **Service Layer** [32] - each business rule lives in one service (for example `authorization`, `task_progress_service`, `notification_service`) regardless of the entry point.
- **Unit of Work** [32] - one SQLAlchemy session per request; a domain change, its audit entry, its job and its event commit together.
- **Strategy through protocols** [33] - `VectorStore` with `PgVectorStore`, `PrivateStorage` with a local implementation, and a push provider base; tests substitute fakes without touching callers.
- **Registry / table-driven dispatch** - 20 declared AI tools, 5 declared agents, ingestion processors and job handlers; anything not registered is unreachable.
- **State machine as transition table** - `VERSION_TRANSITIONS`, the ingestion `TRANSITIONS` table and the voice command transitions reject illegal state changes centrally.
- **Publish-subscribe** - domain events feed agents and the realtime publisher without coupling producers to consumers.
- **Transactional outbox (simplified)** - jobs and notifications are written in the causing transaction and acted on only after commit.
- **Interceptor** - the web and mobile HTTP clients attach tokens and perform single-flight refresh in one place.

Chapter 5: Implementation and Testing

This chapter describes the tools used, explains the technically significant modules from the code, describes version control, and reports the testing strategy and the results of running the automated suites for this report. Simple create-read-update-delete endpoints are not described in detail.

5.1 Development Environment and Tools

Table 5.1: Verified technology choices (versions from the dependency manifests)

Area	Technology	Version / note
Backend	Python, FastAPI, Uvicorn	Python 3.11 image; FastAPI 0.104.1; Uvicorn 0.24
Persistence	PostgreSQL, SQLAlchemy, Alembic, pgvector	PostgreSQL 15; SQLAlchemy 2.0.23; Alembic 1.12.1; pgvector 0.3.6
Security	python-jose (JWT), passlib (Argon2), Pydantic	HS256 tokens; Pydantic 2.5
AI and documents	OpenAI SDK, tiktoken, pypdf	embedding and answer models configurable; defaults text-embedding-3-small and gpt-4.1-mini
BIM	IfcOpenShell	0.8.5
Web client	React, TypeScript, Vite, Tailwind CSS, TanStack Query, Zustand	React 19.2; TypeScript 6.0; Vite 8
Mobile client	Flutter, Dart, Riverpod, go_router, Dio	Flutter 3.32.2; Dart 3.8.1
Testing	pytest, Vitest with Testing Library, flutter_test	pytest 8.3.4; Vitest 3.2
Runtime	Docker Compose, nginx	five services (one behind a profile)

Development took place on Windows 11 with Docker Desktop for the database, API and worker, Node.js for the web client and the Flutter SDK for the mobile client. The backend image is built rather than bind-mounted, so new code is tested by mounting the source into a throwaway container.

5.2 Implementation Details, Key Algorithms, and Code Structure

Constructs separates **deterministic logic** from **language-model behaviour** deliberately. Only six modules call the OpenAI API: four in the voice pipeline (transcription, structured analysis, action analysis and response phrasing) and two in RAG (embeddings and answering). Everything else described below - permissions, scheduling, IFC comparison and interference, question routing, the five agents and the alignment score - is deterministic code whose output can be

reproduced and checked. A configuration key for IFC analysis with a model exists but is not used by any module.

5.2.1 Permission resolution

Purpose. Decide what a person may do in a given project. **Input:** user, permission code, optional project. The permission catalogue declares 59 codes, each with default roles and flags (`project_scoped`, `admin_locked`, `office_only`, `never_external`). **Processing** (Listing 5.1): role permissions are resolved, overrides applied, administrator lockout prevented and the external ceilings enforced last. **Output:** a set of codes; FastAPI dependencies raise 403 when the required code is absent, and `explain_permissions` reports which rule decided each code for the Access Control page. **Testing:** 437 tests in the RBAC and authorization group, including dedicated suites for external-party isolation and security regressions.

Listing 5.1: Effective permission resolution (abridged from services/authorization.py)

```
def effective_permissions(db, user, project_id=None) -> set[str]:
    if user.status != UserStatus.ACTIVE:
        return set()
    granted = rbac.resolved_permissions(db, user, project_id)    # office + project role
    granted = _apply_user_overrides(db, user, project_id, granted)
    if rbac.holds_office_admin_role(db, user):                   # lockout protection
        granted |= {item.code for item in CATALOGUE if item.admin_locked}
    if rbac.is_external_participant(db, user, project_id):       # ceilings, after overrides
        granted -= rbac.NON_PROJECT_SCOPED
        granted -= rbac.NEVER_EXTERNAL
    ...                                                          # project reachability filter
    return granted
```

5.2.2 Critical path calculation

`cpm_service.calculate_dependency_cpm` orders tasks topologically with Kahn's algorithm, sorting ready tasks by display order and name so results are deterministic, and rejects cyclic graphs. Each dependency type is converted into a start offset (Listing 5.2), so one forward and one backward pass handle all four types with lags. Total float is LS - ES. Rather than marking every zero-float task, the algorithm reconstructs one connected driving path by walking backwards from the zero-float task that finishes last, following only tight constraints; a long task without dependencies is never reported as a critical path. `test_cpm_service` covers the four types, lags and cycle detection.

Listing 5.2: Dependency-type conversion and forward pass (abridged from services/cpm_service.py)

```
def _start_offset(dependency, predecessor, successor) -> int:
    """Convert a precedence relationship into a successor-start constraint."""
    lag = int(dependency.lag_days or 0)
    relation = _dependency_type(dependency)
    if relation == "start_to_start":
        return lag
    if relation == "finish_to_finish":
        return _duration(predecessor) + lag - _duration(successor)
    if relation == "start_to_finish":
        return lag - _duration(successor)
    return _duration(predecessor) + lag      # finish_to_start (default)

for task_id in order:                          # topological order, cycles rejected
    constraints = [earliest_start[p] + _start_offset(dep, by_id[p], by_id[task_id])
                   for p, dep in predecessors[task_id]]
    earliest_start[task_id] = max(0, max(constraints, default=0))
    earliest_finish[task_id] = earliest_start[task_id] + _duration(by_id[task_id])
```

5.2.3 Durable job queue

IFC processing and geometry, file ingestion, RAG indexing and agent analysis run through `services/jobs`. Seven job kinds are registered. Listing 5.3 shows how a worker claims work. The handler separates a **bad input** (recorded as failed on the domain row; the job completes and is not retried) from a **broken run** (raised, then retried with a 30 s x 2^n backoff up to three attempts). A heartbeat is written every 15 seconds and a reaper returns jobs whose heartbeat is older than 120 seconds to the queue. `test_job_runner` pins these guarantees.

Listing 5.3: Claiming the next job under a fleet-wide ceiling (simplified from services/jobs/queue.py)

```
SELECT pg_advisory_xact_lock(:claim_key);                -- serialise claimers
SELECT count(*) FROM background_jobs WHERE queue = :q AND status = 'RUNNING';
-- refuse when the fleet-wide ceiling is reached, otherwise:
UPDATE background_jobs
   SET status = 'RUNNING', attempts = attempts + 1, locked_by = :worker,
       started_at = now(), heartbeat_at = now()
 WHERE id = (SELECT id FROM background_jobs
             WHERE queue = :q AND status = 'QUEUED' AND run_after <= now()
             ORDER BY run_after, created_at
             FOR UPDATE SKIP LOCKED LIMIT 1)
RETURNING id, kind, payload, attempts;
```

5.2.4 File ingestion

Input: an uploaded file. **Validation:** size, extension and content signature; SHA-256 checksum; duplicate detection. **Processing:** classification into a file category, then dispatch to a registered processor - PDF, Office documents, IFC (bridged into the IFC pipeline), ZIP packages whose members become child files, and other types. **Output:** READY, PARTIAL or FAILED with extracted metadata. PARTIAL means the file is usable but a capability was unavailable, such as

a scanned PDF without text or a DWG that is stored only. When a file with text settles and automatic indexing is on (the default), an indexing job is queued. **Failure handling:** every state change passes the transition table (Figure 3.7); a retry re-enters the queue.

5.2.5 Retrieval-augmented answering

Indexing splits text into chunks of about 500 tokens with a 75-token overlap, embeds them and stores them in `document_chunks.embedding vector(1536)` with an HNSW cosine index. The classes involved are shown in Figure 5.1: `retrieve()` depends on the `VectorStore` protocol, implemented by `PgVectorStore` with pgvector's cosine-distance operator so that ranking and the top-k limit (default 5) run inside PostgreSQL.

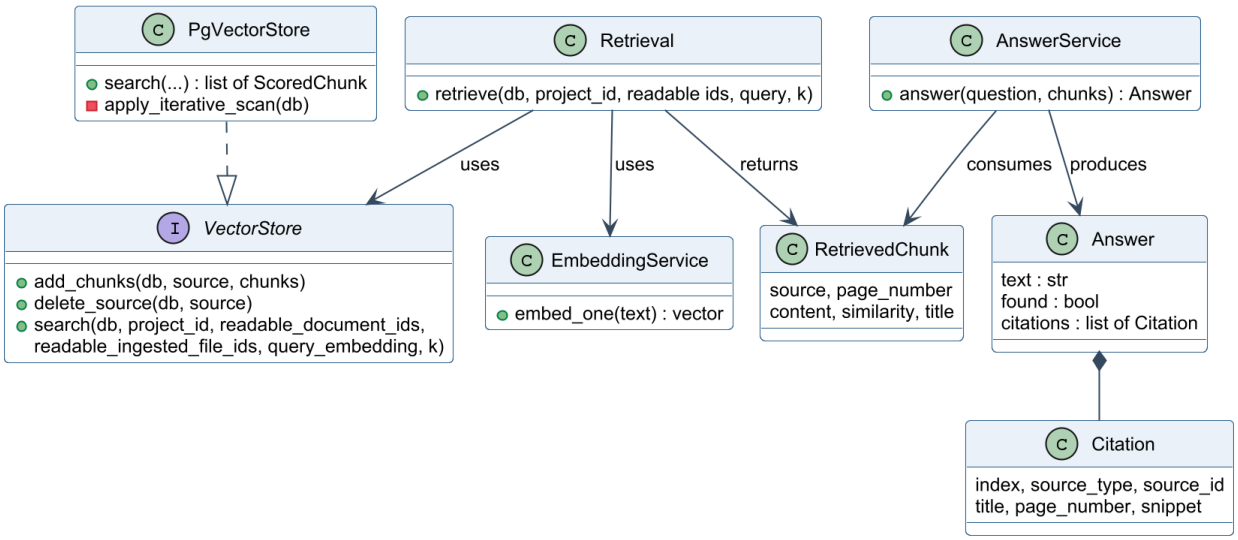


Figure 5.1: Class diagram - retrieval and answering services

Grounding is enforced by three mechanisms that do not depend on the model behaving well. The system prompt forbids outside knowledge and requires the sentinel token `INSUFFICIENT_CONTEXT` when the passages do not support an answer. Citations are constructed from the retrieved chunk records, never parsed from the model's text, so a citation cannot point to a page that was not retrieved. And `found` is computed: when nothing readable is retrieved, no model call is made at all. Before retrieval, `knowledge_router` sends questions about progress, site reports or model findings to structured data instead, because a similarity search over PDFs cannot answer them.

5.2.6 IFC processing, revision comparison and interference

Processing. The parser runs in a separate process with a timeout and an address-space cap, so a malformed model cannot exhaust the API or worker. It extracts the spatial hierarchy, elements with GlobalId, class, discipline, properties, quantities and materials, and runs quality checks;

each status change passes `VERSION_TRANSITIONS` (Figure 3.6). Processing also creates task suggestions per element class (for example footings to "Foundation installation") and per-storey milestone suggestions, each marked with its duplicate risk.

Comparison (Listing 5.4, Figure 4.13). Elements of both versions are keyed by GlobalId. Content changes are detected by comparing stable hashes of properties and quantities, placement and storey/space location, geometry and identity fields. When elements lack stable GlobalIds, the comparison is flagged LOW confidence instead of silently mis-reporting identity changes as additions and removals. The comparison runs synchronously within the request.

Listing 5.4: Revision comparison and impact logic (pseudocode of ifc_processing_service.compare_versions)

```
base_items   = {e.global_id: e for e in elements_of(base)}
target_items = {e.global_id: e for e in elements_of(target)}
for gid in base_items.keys() - target_items.keys():   changes.append(REMOVED)
for gid in target_items.keys() - base_items.keys():   changes.append(ADDED)
for gid in base_items.keys() & target_items.keys():
    old, new = base_items[gid], target_items[gid]
    if delta(old.properties, new.properties) or delta(old.quantities, new.quantities) \
       or location_delta(old, new) or old.geometry_hash != new.geometry_hash or identity_delta:
        record = MODIFIED (MOVED if location changed)
        record.severity = HIGH if old.entity_type in {IfcColumn, IfcBeam, IfcSlab, IfcFooting} \
                          and (geometry or location changed) else MEDIUM
# impact: linked items -> confidence 1.0; otherwise up to 20 same-discipline tasks -> 0.65
```

Interference. Geometry generation records an axis-aligned bounding box per element. `detect_interferences` compares structural elements with service elements using a spatial grid to limit comparisons, converts lengths with the model's declared unit and refuses to run when the unit is unknown, and reports what it analysed and why analysis was limited, because "no findings" and "could not look" must not look the same. Severity and confidence follow from penetration depth; results are stored as findings that can become issues. Bounding boxes over-approximate real shapes, so these are candidates for review, not confirmed clashes.

5.2.7 Model-to-execution alignment score

`ai_insight_engine.calculate_alignment` computes a transparent score from counts: the share of modelled elements whose discipline is covered by tasks (weight 40%), discipline coverage (25%), tasks linked to model locations (15%), issues linked to model elements (10%) and verified evidence linked to the model (10%). The weights are returned with the score so that a user can see how it was computed. The score is a design aid; its weights have not been calibrated against project outcomes.

5.2.8 Voice assistant and agents

The voice pipeline stores audio privately, transcribes it and requests a structured analysis from the model. Routing between a question and a change is deterministic and corroborates several independent signals, so one imperfect field cannot turn a question into a modification; an empty action list never becomes an action. A change becomes drafts with a ten-minute confirmation token; on confirmation, permissions are re-checked and each action is executed and recorded with an audit entry, its before and after state and a domain event. The five agents are rule-based analyzers over permission-checked tools; each finding carries evidence a reader can re-check, and each agent runs in isolation so that one failure does not affect the others.

5.3 Version Control and Change Management

The project is versioned with Git and hosted on GitHub. The current branch history contains 38 commits between 2026-07-24 and 2026-10-09, organised in feature and fix branches merged through pull requests (for example the RBAC migration and the single start sequence). Schema changes are Alembic migrations (71 files) applied automatically at container start. The RBAC redesign was staged: the new model ran beside the old one, an equivalence comparison was run for every account, and only then were the retired columns dropped. The working branch currently carries a large set of uncommitted changes; this report describes and tests that working tree.

5.4 Testing Strategy

Testing follows the test pyramid [46]: many fast tests of pure logic, a large layer of API-level integration tests against a real database, and a few system-level checks.

5.4.1 Unit Testing

Unit tests exercise pure functions and contracts: CPM, schedule-date arithmetic, ingestion transition tables and classification, chunking, AI tool and agent contracts, voice routing and rules, and interference geometry on synthetic elements. On the clients, Vitest with Testing Library tests components such as modal focus behaviour and permission guards, and `flutter_test` tests Dart logic such as voice outcome semantics and audio format detection.

5.4.2 Integration Testing

Most backend tests call FastAPI endpoints through a test client with real tokens and a real PostgreSQL database with pgvector, and assert both responses and database state. Dedicated suites cover external-party isolation, authorization equivalence, AI tool authorization, file

download authorization, the job runner, realtime delivery and IFC failure paths such as corrupt files, worker crashes and oversized results.

5.4.3 System Testing

System-level verification consists of `test_start_sequence` (every environment uses the same start sequence), whole-workflow tests such as `test_ifc_workflow` and `test_voice_pipeline_wiring`, and a manual QA plan (`docs/MANUAL_QA_TEST_PLAN.md`) with prioritised smoke, authentication, project, task, document, IFC and AI cases. The manual plan's result boxes are unticked, so no manual system-test results are claimed.

5.5 Test Results and Analysis

The three automated suites were executed on 24 October 2026 against the current working tree; no source file changed between the runs and the writing of this report. The backend suite ran in a throwaway container against a separate, freshly migrated `pgvector/pgvector:pg15` database, so the development database was not touched, and no OpenAI key was provided. Logs and the JUnit file are in the `evidence` folder. Table 5.2 groups the backend results by functional area, assigned by test-module name.

Table 5.2: Backend test results by functional area (pytest)

Area	Modules	Tests	Passed	Failed	Skipped
Authentication and security	18	298	298	0	0
RBAC and authorization	19	437	437	0	0
IFC / BIM	14	179	178	0	1
Ingestion, RAG and knowledge	16	832	831	1	0
Voice, AI tools and agents	25	675	675	0	0
Projects, tasks, scheduling and field	17	222	222	0	0
Messaging, notifications, realtime, jobs	8	155	155	0	0
Total	117	2798	2796	1	1

Table 5.3: Test outcomes by status across all suites

Status	Count	Details
Executed and passed	2796 + 294 + 141	backend + web (full parallel run) + mobile
Executed and failed	1 + 16	backend: one test needing a live OpenAI key; web: timeouts under load, all 65 tests in the affected files passed when re-run serially
Skipped	1	IFC worker-crash test that runs only when explicitly enabled
Blocked by environment	1	the backend failure above; not re-run with a real key, to avoid sending data to a paid external service
Not executed	-	manual QA plan; live AI-provider calls; push delivery through Firebase; performance, load and security penetration tests

The backend suite collected 2798 test cases (the 1964 test functions expand through parametrisation) and took about 15 minutes. The single failure, `test_rag_ingested_files::test_the_background_job_indexes_and_closes_its_own_session`, raised "OPENAI_API_KEY is not configured on this server": unlike neighbouring tests, it calls the real embedding service. It is an environment-dependent failure, not a pass, and it also reveals a test-isolation weakness - the test should inject a fake embedding service.

In the web suite, 294 of 310 tests passed in the full parallel run; all 16 failures were interaction tests exceeding the default 5-second timeout while the mobile suite ran on the same machine, and the 65 tests in the nine affected files all passed when re-run serially. This indicates machine load rather than defects, but also that several tests are close to their timeout. The mobile suite passed all 141 tests.

What these results do and do not show. They verify behaviour at unit and API-integration level, including the authorization rules, the state machines and the design-to-execution mechanisms. They do not demonstrate that complete user journeys work in the browser or on a phone, that answers from the live model are accurate, or that the system meets any performance target. Code coverage was not measured, so no coverage figure is claimed.

Chapter 6: Business Analysis and Project Evaluation

This chapter analyses how Constructs could create value for its users using the Business Model Canvas [47], discusses costs and impacts qualitatively, and evaluates the project against its objectives. No commercial deployment, customer or partnership exists, so the business analysis is prospective and contains no invented figures.

6.1 Business Model Canvas

6.1.1 Key Partners

A hosting provider for the containers and database; an AI model provider for transcription, embeddings and answers (replaceable, because model access is isolated behind service classes); Firebase for push notifications; and, for validation, universities and engineering associations able to host pilot projects.

6.1.2 Key Activities

Completing the partial features, security maintenance, onboarding each office (configuring roles, disciplines and templates) and supporting users.

6.1.3 Value Propositions

- One project record that external parties can safely join, with permissions the office controls and contractors cannot use to certify their own work.
- Explicit, reviewable links from model revisions to the tasks, issues and milestones they may affect.
- Answers from contracts and specifications with citations that can be checked.
- Field reporting by voice in Arabic or English, confirmed before it changes the plan.
- A self-hosted deployment that an office can operate itself.

6.1.4 Customer Relationships

Assisted onboarding, because every office configures its own roles and processes, followed by support and periodic updates.

6.1.5 Customer Segments

Primary: small and medium consulting engineering offices that supervise building projects with several external parties. Secondary users - clients and contractors - join through the office's projects.

6.1.6 Key Resources

The code base and its automated test suite, the permission catalogue and role templates, the documentation and the domain knowledge encoded in workflows such as task review and IFC coordination.

6.1.7 Channels

Direct relationships with offices, demonstrations, professional and academic networks, and the Android and iOS app stores (an iOS release requires a macOS build environment).

6.1.8 Cost Structure

Development and maintenance effort, hosting (compute, database, storage), variable AI usage and small costs for push and e-mail; Section 6.2 discusses them.

6.1.9 Revenue Streams

Options for future validation rather than tested pricing: a subscription per office or per active project, a usage-based tier for AI features whose cost scales with use, and installation and support contracts for offices that host the system themselves.

6.2 Cost Structure and Budget Analysis

No money was spent on software licences: every framework and tool used is open source or free for development, and the development machine was the author's own. The only potentially paid service used was the OpenAI API, whose usage was not metered for this report. Operating costs would consist of:

- **Fixed and recurring effort** - development, maintenance and support; not time-tracked during the project.
- **Hosting** - compute for the API, worker and web server, plus database and file storage; driven by the number of offices, projects, documents and models. Not measured, since only local deployment exists.

- **AI usage** - variable, driven by tokens for transcription, embeddings and answers. Not metered, but every provider call is logged in `ai_provider_calls` with model, latency and outcome, so a pilot could measure it exactly.
- **Push, e-mail and store accounts** - low, driven by notification volume and publication.

A quantitative budget would require at least a pilot deployment with a known number of users and projects; the AI cost should be measured from the call log before any pricing decision.

6.3 Impact Analysis

6.3.1 Social Impact

Arabic interfaces and spoken-Arabic input lower the barrier for site staff who are more comfortable speaking than typing in English. Permission-controlled sharing gives owners direct visibility of progress and evidence, which can strengthen trust between parties, and every change remains traceable to a person rather than to an AI.

6.3.2 Economic Impact

Expected benefits are less time spent searching for and re-entering information, earlier recognition of schedule risk through dependency-driven CPM and blocker tracking, earlier recognition of model changes that affect planned work, and fewer disputes thanks to an auditable record. These are consistent with the productivity levers identified for the sector [5] but have not been measured; no savings figure is claimed.

6.3.3 Environmental Impact

Digital reporting can reduce printing and some site visits, and recognising coordination problems before construction can reduce rework and waste, a benefit generally associated with BIM [16]. Hosting and AI inference consume energy. No environmental measurement was performed.

6.4 Project Evaluation and Lessons Learned

6.4.1 Achievement of objectives

- **O1 - connected project record:** achieved (FR-06 to FR-14, FR-24 to FR-29).
- **O2 - configurable, safe access:** achieved and strongly tested (FR-03 to FR-05; 437 passing RBAC tests).
- **O3 - schedule control:** achieved (FR-09 to FR-11).
- **O4 - design-to-execution links:** largely achieved (FR-15 to FR-18); the 3D viewer is in transition (FR-19).

- **O5 - document-grounded assistance:** achieved at code level (FR-20, FR-21); answer quality with the live model has not been evaluated.
- **O6 - field feedback:** achieved (FR-24, FR-25); not trialled on a real site.
- **O7 - traceability:** achieved (FR-29); push delivery requires Firebase configuration.

6.4.2 Strengths

- One centrally tested authorization path with ceilings that configuration cannot override.
- AI constrained by architecture: deterministic routing, record-built citations, mandatory confirmation and full action history.
- Design-to-execution links that are explicit, reviewable and audited rather than automatic.
- Correct behaviour with several API workers through database-level locking and transactional fan-out.
- A large backend regression suite (2796 passing tests) executed for this report.

6.4.3 Limitations

Partially implemented:

- Budget and expense tracking linked to evidence (FR-08).
- 3D model viewing, removed from the current working tree (FR-19).

Implemented but not yet verified end to end:

- Complete user journeys in the browser and on a phone; the manual QA plan has not been executed.
- Answer quality of RAG and voice with the live model; no evaluation set exists.
- Push delivery through Firebase in a real deployment.
- Performance, load behaviour and security under attack; no measurements or penetration tests exist.

Design limitations and dependencies:

- Interference detection uses bounding boxes and over-approximates shapes; the discipline fallback for impact suggestions is a heuristic with a fixed confidence of 0.65.
- Uploaded files are stored in host-local volumes, so the backend cannot yet scale horizontally; only a local storage implementation exists.
- The web client keeps tokens in localStorage, and one signing key serves every token type without rotation.
- Docker Compose defaults are development-grade (default database password, published database port); no CI pipeline exists.

- AI features depend on the OpenAI API; one backend test depends on a live key.

6.4.4 Future work

1. Execute the manual QA plan and add automated browser tests for the main journeys.
2. Build an evaluation set of project questions with expected citations to measure retrieval and answer quality.
3. Decide on the 3D viewer and complete expense tracking linked to evidence.
4. Move refresh tokens to HttpOnly cookies, add signing-key identifiers, and implement S3-compatible storage.
5. Add CI with coverage reporting, and run a pilot with a consulting office to measure usability, AI cost and time saved.

6.4.5 Lessons learned

Authorization must be one function with tests that attack it, because a permission check copied into many places eventually diverges. Using the database for queueing, locking and messaging is a sound choice for a small team when each mechanism is chosen for correctness under several processes. AI features are trustworthy only when their guarantees are enforced in code - making citations impossible to fabricate and confirmation mandatory proved more reliable than instructing the model - and many problems that look like AI problems, such as routing a question or detecting a schedule risk, are better solved deterministically. Finally, documentation drifts: several statements in the project's own documents were out of date, which is why this report was written from the code.

Appendices

Appendix A: Glossary of Terms

- **AABB** - Axis-aligned bounding box: the smallest box aligned with the coordinate axes that encloses an element.
- **BIM** - Building information modelling: an object-based digital representation of a facility.
- **CPM** - Critical path method: scheduling based on earliest and latest times and float.
- **Embedding** - A numeric vector representing the meaning of a text passage, used for similarity search.
- **FS / SS / FF / SF** - Finish-to-start, start-to-start, finish-to-finish and start-to-finish dependency types.
- **GlobalId** - The globally unique identifier that IFC assigns to each object.
- **HNSW** - Hierarchical navigable small world graph, an approximate nearest-neighbour index.
- **IFC** - Industry Foundation Classes, the open BIM exchange schema (ISO 16739-1).
- **JWT** - JSON Web Token, a signed token carrying claims (RFC 7519).
- **MCP** - Model Context Protocol, through which external AI clients call tools (disabled by default).
- **RAG** - Retrieval-augmented generation: answering from retrieved passages.
- **RBAC** - Role-based access control.
- **RFI** - Request for information about design or contract documents.
- **RTL** - Right-to-left text direction, used for Arabic.
- **SSE** - Server-sent events: one-way streaming from server to browser.
- **Step-up** - Additional verification required before a sensitive operation.

Appendix B: References

[1] S. Ramirez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[2] A. Kane and contributors, "pgvector: Open-source vector similarity search for Postgres," GitHub repository. [Online]. Available: https://github.com/pgvector/pgvector

[3] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev/

[4] Google LLC, "Flutter documentation." [Online]. Available: https://docs.flutter.dev/

[5] McKinsey Global Institute, "Reinventing construction: A route to higher productivity," McKinsey & Company, Feb. 2017. [Online]. Available: https://www.mckinsey.com/capabilities/operations/our-insights/reinventing-construction-through-a-productivity-revolution

[6] W. Shen *et al.*, "Systems integration and collaboration in architecture, engineering, construction, and facilities management: A review," *Advanced Engineering Informatics*, vol. 24, no. 2, pp. 196-207, 2010, doi: 10.1016/j.aei.2009.09.001.

[7] M. Oraee, M. R. Hosseini, D. J. Edwards, H. Li, E. Papadonikolaki, and D. Cao, "Collaboration barriers in BIM-based construction networks: A conceptual model," *International Journal of Project Management*, vol. 37, no. 6, pp. 839-854, 2019, doi: 10.1016/j.ijproman.2019.05.004.

[8] R. Sacks, C. Eastman, G. Lee, and P. Teicholz, *BIM Handbook: A Guide to Building Information Modeling for Owners, Designers, Engineers, Contractors, and Facility Managers*, 3rd ed. Hoboken, NJ, USA: Wiley, 2018, ISBN 978-1-119-28753-7.

[9] International Organization for Standardization, *ISO 16739-1:2018 Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries - Part 1: Data schema*, Geneva, Switzerland, 2018 (withdrawn 2024; superseded by ISO 16739-1:2024).

[10] J. E. Kelley and M. R. Walker, "Critical-path planning and scheduling," in *Proc. Eastern Joint IRE-AIEE-ACM Computer Conf.*, Boston, MA, USA, Dec. 1959, pp. 160-173, doi: 10.1145/1460299.1460318.

[11] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38-47, Feb. 1996, doi: 10.1109/2.485845.

[12] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1-38, 2023, doi: 10.1145/3571730.

[13] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. arXiv:2005.11401.

[14] IfcOpenShell contributors, "IfcOpenShell - The open source IFC toolkit and geometry engine." [Online]. Available: https://ifcopenshell.org/

[15] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA, USA: PMI, 2021, ISBN 978-1-62825-664-2.

[16] S. Azhar, "Building information modeling (BIM): Trends, benefits, risks, and challenges for the AEC industry," *Leadership and Management in Engineering*, vol. 11, no. 3, pp. 241-252, 2011, doi: 10.1061/(ASCE)LM.1943-5630.0000127.

[17] M. Awe, A. Malhi, M. Budka, N. Mavengere, and B. Dave, "Towards 4D BIM: A systematic literature review on challenges, strategies and tools in leveraging AI with BIM," *Buildings*, vol. 15, no. 7, p. 1072, 2025, doi: 10.3390/buildings15071072.

[18] H. Wefki, M. Elnahla, and E. Elbeltagi, "BIM-based schedule generation and optimization using genetic algorithms," *Automation in Construction*, vol. 164, p. 105476, 2024, doi: 10.1016/j.autcon.2024.105476.

[19] O. Akponeware and Z. A. Adamu, "Clash detection or clash avoidance? An investigation into coordination problems in 3D BIM," *Buildings*, vol. 7, no. 3, p. 75, 2017, doi: 10.3390/buildings7030075.

[20] A. Saka *et al.*, "GPT models in construction industry: Opportunities, limitations, and a use case validation," *Developments in the Built Environment*, vol. 17, p. 100300, 2024, doi: 10.1016/j.dibe.2023.100300.

[21] A. Erfani and A. Mansouri, "Applications of multimodal large language models in construction industry," *Advanced Engineering Informatics*, vol. 69, p. 103909, 2026, doi: 10.1016/j.aei.2025.103909.

[22] J. Zheng and M. Fischer, "Dynamic prompt-based virtual assistant framework for BIM information search," *Automation in Construction*, vol. 155, p. 105067, 2023, doi: 10.1016/j.autcon.2023.105067.

[23] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, Dec. 2023.

[24] OpenAI, "Vector embeddings," *OpenAI Platform Documentation*. [Online]. Available: https://platform.openai.com/docs/guides/embeddings

[25] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[26] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Information and System Security*, vol. 4, no. 3, pp. 224-274, 2001, doi: 10.1145/501978.501980.

[27] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[28] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[29] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," IETF RFC 6749, Oct. 2012.

[30] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[31] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 memory-hard function for password hashing and proof-of-work applications," IETF RFC 9106, Sep. 2021.

[32] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002, ISBN 978-0-321-12742-6.

[33] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994, ISBN 978-0-201-63361-0.

[34] PostgreSQL Global Development Group, "SELECT - The locking clause," *PostgreSQL 15 Documentation*. [Online]. Available: https://www.postgresql.org/docs/15/sql-select.html

[35] PostgreSQL Global Development Group, "Explicit locking - Advisory locks," *PostgreSQL 15 Documentation*. [Online]. Available: https://www.postgresql.org/docs/15/explicit-locking.html

[36] PostgreSQL Global Development Group, "NOTIFY," *PostgreSQL 15 Documentation*. [Online]. Available: https://www.postgresql.org/docs/15/sql-notify.html

[37] WHATWG, "HTML Living Standard - Server-sent events." [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html

[38] Procore Technologies, Inc., "Procore - Construction management software," product website. [Online]. Available: https://www.procore.com/

[39] Autodesk, Inc., "Autodesk construction solutions (Autodesk Construction Cloud / Forma)," product website. [Online]. Available: https://construction.autodesk.com/

[40] Oracle Corporation, "Oracle Aconex," product website. [Online]. Available: https://www.oracle.com/construction-engineering/aconex/

[41] Oracle Corporation, "Primavera P6 Enterprise Project Portfolio Management," product website. [Online]. Available: https://www.oracle.com/construction-engineering/primavera-p6/

[42] Procore Technologies, Inc., "View coordination issues in the model viewer," *Procore Support*. [Online]. Available: https://support.procore.com/products/online/user-guide/project-level/models/tutorials/view-coordination-issues-in-the-model-viewer

[43] Autodesk, Inc., "About Model Coordination," *Autodesk Help*. [Online]. Available: https://help.autodesk.com/cloudhelp/ENU/Coord-GS/files/About_Model_Coord.html

[44] Procore Technologies, Inc., "More about Assist," *Procore Support*. [Online]. Available: https://support.procore.com/products/online/user-guide/project-level/assist/more-about-assist

[45] Autodesk, Inc., "GCs, meet Autodesk Assistant - a smart tool that can help you find answers," *Autodesk Construction Blog*. [Online]. Available: https://www.autodesk.com/blogs/construction/gcs-meet-autodesk-assistant-a-smart-tool-that-can-help-you-find-answers/

[46] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Upper Saddle River, NJ, USA: Addison-Wesley, 2010, ISBN 978-0-321-57936-2.

[47] A. Osterwalder and Y. Pigneur, *Business Model Generation*. Hoboken, NJ, USA: Wiley, 2010, ISBN 978-0-470-87641-1.

Appendix C: Supporting Materials

Feature switches. Optional behaviour is controlled by settings in `backend/app/core/config.py`: `RAG_ENABLED` (false in code, true in docker-compose), `RAG_AUTO_INDEX_ENABLED` (true), `VOICE_FEATURE_ENABLED` (false), `AGENT_AUTO_ANALYSIS_ENABLED` (false), `PUSH_ENABLED` (false), `REALTIME_ENABLED` (true), `MCP_ENABLED` (false), `JOB_RUNNER_EMBEDDED` (true; false under Compose, where the worker service runs jobs) and `IFC_MAX_CONCURRENT_PROCESSING` (2).

Diagram sources. Every diagram is provided as editable source in `diagrams/src` (PlantUML `.puml` files and one Python script) with its rendered image in `diagrams/png`; `diagrams/README.md` maps each figure number to its files. **Evidence.** Raw test logs and the JUnit file are in `evidence`.

Appendix D: Requirements Traceability Summary

Table A.1 lists each functional requirement with a representative automated test and its status. The complete matrix, including all test modules, implementation files, related figures and the non-functional requirements, is provided in Constructs_Traceability_Matrix.xlsx.

Table A.1: Functional requirements with representative tests

ID	Requirement	Representative test	Status
FR-01	Sign-in and sessions	test_auth_refresh_concurrency (+more)	Implemented
FR-02	Step-up verification	test_step_up_security	Implemented
FR-03	Configurable roles	test_rbac_customization (+more)	Implemented
FR-04	Per-person exceptions	test_authorization (+more)	Implemented
FR-05	External-party isolation	test_external_party_isolation (+more)	Implemented
FR-06	Projects	test_project_deletion_authorization (+more)	Implemented
FR-07	Members and parties	test_project_team_assignment	Implemented
FR-08	Dashboards and budget view	test_action_center_counts (+more)	Partial
FR-09	Task management	test_task_update_reliability (+more)	Implemented
FR-10	Task execution and review	test_consultant_approval_policy (+more)	Implemented
FR-11	Dependencies and CPM	test_cpm_service (+more)	Implemented
FR-12	Document library	test_file_download_authorization (+more)	Implemented
FR-13	File ingestion	test_ingestion_pipeline (+more)	Implemented
FR-14	Design changes and requests	test_authorization (+more)	Implemented
FR-15	IFC processing	test_ifc_parser (+more)	Implemented
FR-16	Model-derived suggestions	test_ifc_workflow	Implemented
FR-17	Revision comparison and impact	test_ifc_workflow	Implemented
FR-18	Coordination findings	test_ifc_interference (+more)	Implemented
FR-19	3D viewing	test_ifc_geometry (+more)	Partial
FR-20	Grounded document answers	test_rag_pipeline (+more)	Implemented
FR-21	Question routing	test_knowledge_router (+more)	Implemented
FR-22	Rule-based insights and agents	test_agent_orchestration (+more)	Implemented
FR-23	Event-driven agent analysis	test_agent_autonomous_detection	Implemented, disabled by default
FR-24	Site reports and evidence	test_site_report_verification (+more)	Implemented
FR-25	Voice-to-action	test_voice_command_workflow (+more)	Implemented
FR-26	Issues, visits and reminders	test_site_visit_scheduling (+more)	Implemented
FR-27	Messaging	test_messaging_policy (+more)	Implemented
FR-28	Notifications	test_notification_api (+more)	Implemented
FR-29	Audit and AI history	test_domain_event_emission (+more)	Implemented
FR-30	Bilingual interface	IFCActionTabs.i18n.test.ts (+more)	Implemented

Appendix E: Test Execution Details

- **Backend:** a copy of `backend/` mounted into a container built from the project's backend image plus pytest 8.3.4; a separate `pgvector/pgvector:pg15` container on a private Docker network; `alembic upgrade head`, role bootstrap, then `python -m pytest tests/ --junitxml`; no OpenAI key; `RAG_ENABLED=false` at process level (tests enable it where needed).
- **Web:** `npx vitest run` (Node 22, jsdom); serial re-run of the nine affected files with `--no-file-parallelism`.
- **Mobile:** `flutter test` with the host Flutter SDK.
- **To complete later:** a run with a test OpenAI key for the one environment-dependent test, and execution of the manual QA plan.